

Politechnika Warszawska

W Y D Z I A Ł E L E K T R Y C Z N Y



Instytut Elektrotechniki Teoretycznej i Systemów Informacyjno-Pomiarowych

Praca dyplomowa magisterska

na kierunku Informatyka Stosowana
w specjalności Informatyka w Biznesie

Generator tekstowych danych losowych dla zadanego kontekstu wykorzystujący LLM

inż. Jakub Prokopiuk

numer albumu 319002

promotor

dr inż. Radosław Roszczyk

WARSZAWA 2026

Generator tekstowych danych losowych dla zadanego kontekstu wykorzystujący LLM

Streszczenie

Poniższa praca magisterska poświęcona jest zaprojektowaniu i implementacji zaawansowanego systemu do generowania syntetycznych, relacyjnych zbiorów danych, który łączy tradycyjne metody algorytmiczne z możliwościami dużych modeli językowych.

W ramach pracy opracowano aplikację o architekturze mikroservisowej, wykorzystującą konteneryzację Docker do integracji warstwy frontendowej opartej na bibliotece React, warstwy backendowej zrealizowanej w frameworku FastAPI oraz systemu kolejkowania zadań asynchronicznych Redis i Celery. Takie podejście pozwoliło na zapewnienie skalowalności i wydajności niezbędnej do generowania dużych wolumenów danych bez blokowania interfejsu użytkownika.

Kluczowym elementem rozwiązania jest silnik generujący, który implementuje hybrydowe podejście do tworzenia danych. Wykorzystuje on bibliotekę Faker do danych strukturalnych oraz modele LLM - obejmujące zarówno rozwiązania chmurowe (OpenAI), jak i modele uruchamiane lokalnie - do generowania treści kreatywnych i kontekstowych. System automatycznie analizuje schemat bazy danych i rozwiązuje zależności kluczy obcych w celu zachowania integralności referencyjnej generowanych rekordów.

Zaimplementowane rozwiązanie wykracza poza standardowe generatory plików płaskich, oferując mechanizm bezpośredniego zrzutu danych do popularnych silników bazodanowych, takich jak PostgreSQL, MySQL, MSSQL oraz Oracle. Dodatkowo system wyposażono w moduł uwierzytelniania, zarządzanie projektami w chmurze oraz podgląd postępu generowania w czasie rzeczywistym. Rezultatem pracy jest kompletne, gotowe do wdrożenia narzędzie, wspierające procesy testowania, prototypowania i analityki danych.

Słowa kluczowe: Generowanie danych syntetycznych, LLM, FastAPI, bazy danych, Docker, Sztuczna Inteligencja, Redis, Celery

Textual Data Generator for a Given Context Using LLM

Abstract

This master's thesis is devoted to the design and implementation of an advanced system for generating synthetic relational datasets, which combines traditional algorithmic methods with the capabilities of Large Language Models.

As part of the thesis, an application with a microservice architecture was developed, utilizing Docker containerization to integrate a React-based frontend, a FastAPI backend, and an asynchronous task queuing system based on Redis and Celery. This approach ensured the scalability and performance necessary to generate large volumes of data without blocking the user interface.

A key element of the solution is the generation engine, which implements a hybrid approach to data creation. It employs standard libraries for structured data and Large Language Models—including both cloud-based solutions and local instances—to generate creative and contextual content. The system automatically analyzes the database schema, resolving foreign key dependencies to maintain the referential integrity of the generated records.

The implemented solution extends beyond standard flat-file generators, offering a mechanism for direct data dumps to popular database engines such as PostgreSQL, MySQL, MSSQL, and Oracle. Additionally, the system is equipped with an authentication module, cloud project management capabilities, and real-time generation progress monitoring via WebSocket protocol. The result of the work is a complete and ready-to-deploy tool that supports testing, prototyping, and data analytics processes.

Keywords: Synthetic Data Generation, LLM, FastAPI, Databases, Docker, Artificial Intelligence, Redis, Celery

Spis treści

1	Wstęp	9
2	Istniejące rozwiązania	11
2.1	Rozwiązania algorytmiczne i quasi-losowe	11
2.2	Rozwiązania oparte na modelach generatywnych	12
2.3	Rozwiązania oparte na modelach językowych	12
2.4	Proponowane rozwiązanie	12
2.5	Porównanie rozwiązań	13
3	Analiza i wybór technologii	15
3.1	Język programowania backend: Python	15
3.2	Framework webowy: FastAPI vs Flask vs Django	15
3.3	Warstwa prezentacji: React	16
3.4	Baza danych: PostgreSQL	17
3.5	Przetwarzanie asynchroniczne: Celery + Redis	17
3.6	Konteneryzacja: Docker	18
4	Analiza wymagań i funkcjonalność systemu	19
4.1	Wymagania funkcjonalne	19
4.2	Wymagania нефункционалне	20
5	Metodyka pracy	23
5.1	Wykorzystanie wsparcia sztucznej inteligencji	23
5.2	Wkład autorski i decyzyjność	24
6	Implementacja	25
6.1	Architektura systemu	25
6.2	Implementacja backendowa	26
6.3	Implementacja frontendowa	28
6.4	Implementacja algorytmów	38
6.5	WebSocket	40
6.6	Infrastruktura i konteneryzacja	41

7 Podręcznik użytkownika	43
7.1 Wymagania systemowe i instalacja	43
7.2 Pierwsze kroki: logowanie i interfejs	44
7.3 Poradnik: generowanie bazy danych „Przychodnia Lekarska”	44
7.4 Zaawansowane funkcje	45
8 Testowanie systemu	47
8.1 Testy jednostkowe i integracyjne backendu	47
8.2 Testy manualne interfejsu użytkownika	48
8.3 Analiza wydajności	48
8.4 Testy End-to-End (End-to-End (E2E))	49
8.5 Testy bezpieczeństwa	50
8.5.1 Analiza podatności zależności (Software Composition Analysis (SCA))	50
8.5.2 Statyczna analiza kodu (SAST)	50
9 Wyzwania i problemy implementacyjne	51
9.1 Zarządzanie stanem w środowisku asynchronicznym	51
9.2 Determinizm i unikalność przy zastosowaniu LLM	51
9.3 Rozwiązywanie zależności cyklicznych	52
9.4 Optymalizacja wydajności generowania danych z użyciem LLM	52
9.5 Obsługa błędów sieciowych	54
10 Podsumowanie i wnioski	57
10.1 Osiągnięcia	57
10.2 Dalszy rozwój aplikacji	57
Bibliografia	59
Wykaz skrótów i symboli	59
Spis rysunków	61
Spis tabel	63
Spis załączników	65

Rozdział 1

Wstęp

We współczesnym świecie dane stanowią jeden z najcenniejszych zasobów przedsiębiorstw, napędzając rozwój systemów informatycznych, analityki biznesowej oraz uczenia maszynowego. Jednakże, wraz ze wzrostem wolumenu przetwarzanych informacji, rośnie również ryzyko związane z ich bezpieczeństwem. W dobie rygorystycznych przepisów o ochronie danych osobowych (GDPR) [european_parliament_2016_679] oraz rosnącej złożoności systemów informatycznych, pozyskanie realistycznych danych testowych, które zachowują spójność logiczną i kontekstową, stanowi istotne wyzwanie inżynierii oprogramowania.

Tradycyjne podejście do testowania oprogramowania często opierało się na wykorzystaniu kopii produkcyjnych baz danych, poddanych procesom anonimizacji lub maskowania. Metody te, choć skuteczne w zachowaniu struktury danych, niosą ze sobą ryzyko reidentyfikacji osób fizycznych [deanonymization_risks] oraz są trudne do skalowania w środowiskach Continuous Integration/Continuous Deployment (CI/CD). Z drugiej strony, proste generatory danych losowych (ang. random data generators) często tworzą zbiory niespójne semantycznie - np. generując recenzję produktu, która nie ma związku z jego kategorią, czy adresy niepasujące do kraju użytkownika. Taka niespójność utrudnia testowanie zaawansowanych algorytmów analitycznych i systemów rekomendacyjnych.

Przełomem w dziedzinie generowania treści stało się upowszechnienie dużych modeli językowych (Large Language Model (LLM)). Modele takie jak GPT-4 czy Llama 3 potrafią generować tekst o wysokim stopniu realizmu, uwzględniając zadany kontekst [llm_synthetic_data]. Jednak samo użycie LLM jest procesem kosztownym obliczeniowo i wolnym, co czyni go niepraktycznym przy generowaniu milionów rekordów bazy danych. Istnieje zatem zapotrzebowanie na rozwiązania hybrydowe, łączące szybkość klasycznych algorytmów z kreatywnością sztucznej inteligencji.

Celem niniejszej pracy jest zaprojektowanie i implementacja narzędzia służącego do proceduralnego generowania relacyjnych zbiorów danych. Proponowane rozwiązanie integruje wydajne algorytmy pseudolosowe (biblioteka Faker) do generowania danych strukturalnych oraz modele LLM do tworzenia danych kontekstowych. System został zaprojektowany w architekturze mikroservisowej,

wykorzystując konteneryzację Docker, kolejkę zadań Redis oraz asynchroniczne przetwarzanie, co pozwala na symulację środowisk produkcyjnych o wysokiej złożoności.

Rozdział 2

Istniejące rozwiązania

Rynek narzędzi do generowania danych syntetycznych jest obecnie dynamicznie rozwijającym się obszarem, który można podzielić na trzy główne kategorie: rozwiązania algorytmiczne, rozwiązania oparte na uczeniu maszynowym oraz nowo powstające rozwiązania oparte o modele językowe. Poniższa analiza przedstawia wiodące narzędzia w tych kategoriach, wskazując ich zalety oraz ograniczenia w kontekście generowania spójnych relacyjnie baz danych.

2.1 Rozwiązania algorytmiczne i quasi-losowe

Jeszcze do niedawna były to najpopularniejsze narzędzia do generowania danych syntetycznych. Sztandarowe narzędzia w tej kategorii to m.in.:

Biblioteka Faker

Faker [**fakerjs**] to biblioteka open-source dostępna w wielu językach programowania (Python, JavaScript, Ruby), umożliwiająca generowanie różnorodnych danych losowych, takich jak imiona, adresy, numery telefonów czy dane finansowe.

- **Zalety:** Bardzo wysoka wydajność (generowanie odbywa się w czasie rzeczywistym), brak kosztów (narzędzie jest open-source), determinizm (możliwość odtworzenia danych przy użyciu seed).
- **Wady:** Brak kontekstualnej spójności między generowanymi danymi. Wygenerowana „recenzja produktu” to zazwyczaj zbiór losowych słów z łaciny lub zdań niemających sensu logicznego. Znaczącą wadą jest również brak wbudowanej obsługi złożonych relacji między tabelami (użytkownik musi sam zaprogramować logikę kluczy obcych).

Mockaroo

Mockaroo to popularne narzędzie webowe (Software as a Service (SaaS)), które pozwala definiować schematy danych w GUI i pobierać je jako Comma-Separated Values (CSV)/Structured Query Language (SQL)/JavaScript Object Notation (JSON).

- **Zalety:** Intuicyjny interfejs użytkownika, szeroki zakres typów danych, możliwość eksportu do różnych formatów, wsparcie dla relacji między tabelami za pomocą prostych skryptów pisanych w Ruby.
- **Wady:** W wersji darmowej generowanie limitowane jest do 1000 wierszy. Mimo zaawansowania, nadal opiera się głównie na losowaniu ze słowników. Funkcje sztucznej inteligencji są ograniczone lub dodatkowo płatne.

2.2 Rozwiązania oparte na modelach generatywnych

Ta grupa narzędzi (np. Gretel.ai [**gretel_platform**], Mostly AI [**mostly**]) posiada duże problemy z prywatnością danych. Działają one na zasadzie: „Wgraj nam swoją prawdziwą bazę danych, a my nauczymy model jej schematu i wygenerujemy nową, która wygląda tak samo, ale nie zawiera danych osobowych”.

- **Zalety:** Potrafią generować dane o wysokim stopniu realizmu, zachowując statystyczne właściwości oryginalnego zbioru.
- **Wady:** Wymagają dostępu do rzeczywistych danych, co może naruszać przepisy o ochronie danych osobowych. Wymagają również danych wejściowych, przez co nie nadają się do tworzenia danych do nowo powstających systemów, gdzie nie ma jeszcze żadnej bazy danych, na której można by uczyć model. Koszty licencyjne są wysokie, a skalowalność ograniczona przez infrastrukturę dostawcy.

2.3 Rozwiązania oparte na modelach językowych

Najnowsza kategoria narzędzi do generowania danych syntetycznych wykorzystuje możliwości LLM, takich jak GPT-4 czy Claude. Przykładowo, możemy poprosić model o wygenerowanie danych do tabeli „Recenzje produktów”, podając mu kontekst (np. kategorie produktów, cechy użytkowników).

- **Zalety:** Wysoki realizm tekstowy, kreatywność, rozumienie kontekstu.
- **Wady:** Wysokie koszty operacyjne, ograniczenia szybkości, model może „wymyślić” nieistniejące ID lub zgubić strukturę JSON/SQL przy dużej ilości danych. Przez ograniczenia kontekstowe, generowanie dużych zbiorów danych wymaga podziału na mniejsze partie, co komplikuje zachowanie spójności referencyjnej między tabelami.

2.4 Proponowane rozwiązanie

Analiza powyższych rozwiązań wskazuje na istotną lukę rynkową. Brakuje narzędzia, które:

- nie wymaga danych wejściowych,
- zapewnia wysoki realizm i kontekstowość danych tekstowych,
- gwarantuje ścisłą integralność relacyjną przy dużej skali,

- jest efektywne kosztowo (nie generuje prostych danych typu „data urodzenia” przez drogie LLM).

2.5 Porównanie rozwiązań

Poniższa tabela (Tabela 1) przedstawia porównanie analizowanych narzędzi z rozwiązaniem autorskim.

Cecha	Faker	Mockaroo	Gretel.ai	DataSynth
Generowanie strukturalne	TAK	TAK	TAK	TAK
Generowanie kontekstowe AI	NIE	Ograniczone	TAK	TAK
Obsługa relacji (FK)	Manualna	Skryptowa	Automatyczna	Automatyczna
Prywatność (Lokalne modele)	-	NIE	Częściowo	TAK (Ollama)
Wymaga danych wejściowych	NIE	NIE	TAK	NIE
Koszt	Darmowy	Freemium	Płatny	Darmowy

Tabela 1. Porównanie narzędzi do generowania danych

Rozdział 3

Analiza i wybór technologii

Wybór odpowiednich narzędzi jest kluczowym elementem procesu projektowania każdego systemu informatycznego. Decyzje podjęte na tym etapie mają bezpośredni wpływ na wydajność, skalowalność, bezpieczeństwo oraz łatwość utrzymania aplikacji w przyszłości. W niniejszym rozdziale przedstawiono analizę dostępnych rozwiązań oraz uzasadniono wybór konkretnych technologii wykorzystanych w projekcie.

3.1 Język programowania backend: Python

Język Python został wybrany jako fundament warstwy serwerowej. Decyzja ta podyktowana była kilkoma czynnikami:

1. **Ekosystem AI/ML:** Python jest niekwestionowanym liderem w dziedzinie sztucznej inteligencji. Biblioteki takie jak `openai` czy `pandas` posiadają natywne wsparcie dla tego języka, co znacznie upraszcza integrację z modelami LLM.
2. **Generatory danych:** Dostępność biblioteki `Faker`, która jest standardem w generowaniu danych testowych.
3. **Szybkość prototypowania:** Zwięzła składnia Pythona pozwala na szybkie tworzenie i testowanie nowych funkcjonalności.

3.2 Framework webowy: FastAPI vs Flask vs Django

Do budowy Application Programming Interface (API) rozważano trzy najpopularniejsze frameworki Pythonowe: Django[`django`], Flask[`flask`] oraz FastAPI[`fastapi`]. Różnią się one przede wszystkim filozofią projektową. Django to duży framework, oferujący gotowe rozwiązania (ORM, panel administracyjny, uwierzytelnianie), co jednak wiąże się z dużym narzutem. Flask i FastAPI to mikroframeworki, dostarczające jedynie niezbędnych narzędzi do obsługi żądań HTTP, pozostawiając programiście swobodę w doborze bibliotek (np. do bazy danych).

Cecha	Django	Flask	FastAPI
Wydajność	Średnia	Średnia	Wysoka
Obsługa Async	Częściowa	Ograniczona	Natywna
Typowanie	Opcjonalne	Brak	Wymuszone (Pydantic)

Tabela 2. Porównanie frameworków webowych Python

Wybrano FastAPI ze względu na natywną obsługę asynchroniczności (kluczowe przy długotrwałych zapytaniach do LLM) oraz automatyczną walidację danych przy użyciu modeli Pydantic, co znacząco podnosi bezpieczeństwo typów.

3.3 Warstwa prezentacji: React

Dla interfejsu użytkownika zdecydowano się na model oparty na technologii jednej strony (Single Page Application (SPA))[spa], co pozwala na płynne przechodzenie między widokami bez konieczności przeładowywania całej strony. W procesie wyboru technologii rozważano trzy wiodące rozwiązania: Angular[angular], Vue.js[vuejs] oraz React [react].

Aby dokonać obiektywnego wyboru, zdefiniowano następujące kryteria porównawcze:

1. **Próg wejścia:** Poziom trudności oraz czas wymagany do efektywnego rozpoczęcia pracy z technologią.
2. **Model danych:** Sposób przepływu danych (jednokierunkowy vs dwukierunkowy) i jego wpływ na przewidywalność stanu aplikacji.
3. **Elastyczność:** Poziom swobody w doborze bibliotek towarzyszących (np. do routingu, zarządzania stanem).

Cecha	Angular	Vue.js	React
Próg wejścia	Wysoki	Niski	Średni
Model danych	Dwukierunkowy	Dwukierunkowy	Jednokierunkowy
Elastyczność	Niska	Wysoka	Wysoka

Tabela 3. Porównanie technologii frontendowych

Analiza alternatyw:

- **Angular:** Jest kompletnym rozwiązaniem typu „wszystko w jednym”. Choć oferuje gotowe narzędzia do większości zadań, narzuca sztywne ramy architektoniczne i wymaga znajomości TypeScript oraz biblioteki RxJS. Dla projektu, w którym priorytetem jest elastyczność i szybkość prototypowania, okazał się zbyt ciężki i skomplikowany.
- **Vue.js:** Pod wieloma względami (np. wydajności) jest zbliżony do Reacta i posiada niższy próg wejścia. Jednak ekosystem Reacta, w tym dostępność gotowych komponentów integrujących

się z nowoczesnymi narzędziami, jest znacznie bogatszy, co zadecydowało o jego odrzuceniu na korzyść Reacta.

Ostatecznie wybrano React, kierując się następującymi atutami:

- **Komponentowość:** Ułatwia budowanie skomplikowanych interfejsów z małych, niezależnych i reużywalnych części (np. formularz pola, modal konfiguracji).
- **Wirtualny Document Object Model (DOM):** Zapewnia wysoką wydajność, minimalizując kosztowne operacje na rzeczywistym drzewie DOM, co jest kluczowe przy wyświetlaniu dużych zestawów wygenerowanych danych.
- **Bogaty ekosystem:** Ogromna społeczność i dostępność gotowych bibliotek (takich jak Lucide React) znacząco przyspiesza proces budowania interfejsu.

Jako narzędzie budujące i środowisko uruchomieniowe wybrano **Vite** [vite], rezygnując z tradycyjnego rozwiązania jakim jest Webpack. Decyzja ta wynikała z fundamentalnych różnic w sposobie działania obu narzędzi:

- **Szybkość startu:** Vite wykorzystuje natywne moduły ES[es_modules] w przeglądarce, co pozwala na natychmiastowe uruchomienie serwera deweloperskiego, niezależnie od rozmiaru aplikacji. Webpack musi zbudować cały kod aplikacji przed startem.
- **Hot Module Replacement (HMR):** W Vite podmiana modułów następuje niemal natychmiastowo, co znacząco przyspiesza tworzenie aplikacji.
- **Optymalizacja kodu:** Vite w trybie produkcyjnym wykorzystuje narzędzie Rollup[rollup], co pozwala na zaawansowane optymalizacje, takie jak *tree-shaking* (usuwanie martwego kodu) oraz *lazy loading* (dzielenie kodu na mniejsze fragmenty ładowane na żądanie). Dzięki temu finalny plik aplikacji jest mniejszy i ładuje się szybciej.

3.4 Baza danych: PostgreSQL

Mimo że generowane dane mogą mieć różną strukturę, zdecydowano się na relacyjną bazę danych PostgreSQL [postgresql] do przechowywania metadanych systemu (użytkownicy, projekty, historia zadań). Głównym argumentem była potrzeba zachowania ścisłej spójności transakcyjnej (ACID [acid_transaction]) przy zarządzaniu projektami oraz natywne wsparcie dla typu danych JSONB. Pozwala to na elastyczne przechowywanie konfiguracji schematów (które są strukturami JSON) przy jednoczesnym zachowaniu zalet języka SQL do zapytań analitycznych.

3.5 Przetwarzanie asynchroniczne: Celery + Redis

Generowanie tysięcy rekordów, szczególnie z użyciem LLM, jest procesem czasochłonnym, który nie może blokować głównego wątku serwera Hypertext Transfer Protocol (HTTP).

- **Celery[celery]:** Jest to rozproszony system kolejkowy, pozwalający na definiowanie zadań, które mogą być wykonywane przez wiele procesów roboczych równolegle.

- **Redis[redis]**: Pełni rolę brokera wiadomości, przechowując informacje o zadaniach do wykonania oraz ich wynikach.

3.6 Konteneryzacja: Docker

Zastosowanie konteneryzacji było wymogiem нефункциональным, mającym na celu zapewnienie powtarzalności środowiska uruchomieniowego [**docker**]. Definicja środowiska w pliku `docker-compose.yml` pozwala na uruchomienie całego stosu (Backend, Frontend, Baza Danych, Redis, Worker, LLM Server) jedną komendą, eliminując problem niezgodności w działaniu na różnych maszynach deweloperskich.

Rozdział 4

Analiza wymagań i funkcjonalność systemu

Proces wytwarzania oprogramowania wymaga precyzyjnego zdefiniowania założeń projektowych. W przypadku systemów wykorzystujących generatywną sztuczną inteligencję, kluczowe jest wyznaczenie granic między deterministycznym działaniem algorytmów a probabilistyczną naturą modeli językowych. Poniższy rozdział definiuje specyfikację funkcjonalną i нефункциональную projektowanego systemu, stanowiącą fundament dla implementacji architektury mikroservisowej opisaney w kolejnych częściach pracy.

4.1 Wymagania funkcjonalne

Głównym celem systemu jest umożliwienie użytkownikom generowania syntetycznych, relacyjnych zbiorów danych o wysokim stopniu realizmu, z wykorzystaniem hybrydowego silnika łączącego metody regułowe i modele LLM.

Definiowanie schematów i typów danych

System powinien udostępniać graficzny interfejs do modelowania struktury bazy danych. Użytkownik powinien mieć możliwość definiowania tabel, kolumn oraz typów danych, wybierając spośród trzech głównych generatorów:

- **Generator deterministyczny:** Powinien służyć do szybkiego tworzenia standardowych danych o przewidywalnej strukturze (np. imiona, adresy, daty, identyfikatory Universally Unique Identifier (UUID)).
- **Generator kontekstowy:** Powinien pozwalać na zdefiniowanie szablonu promptu (np. „Napisz e-mail reklamacyjny dotyczący zamówienia {order_id}”), który następnie zostanie przetworzony przez model językowy w celu wygenerowania unikalnej treści.

- **Generator dystrybucyjny:** Powinien umożliwiać zdefiniowanie ważonych list wartości (np. statusy zamówień z określonym prawdopodobieństwem wystąpienia), aby odzwierciedlić rzeczywisty rozkład danych.

Zachowanie integralności relacyjnej [referential_integrity]

Krytyczną funkcjonalnością systemu jest automatyczne zarządzanie relacjami między tabelami. Aplikacja powinna pozwalać na definiowanie kluczy obcych [foreign_key], a silnik generujący ma za zadanie automatycznie budować graf zależności i ustalać topologiczną kolejność generowania danych. Mechanizm ten musi gwarantować, że rekordy w tabelach podrzędnych będą zawsze odwoływać się do istniejących rekordów w tabelach nadrzędnych, eliminując błędy spójności.

Orkiestracja zadań generowania

Ze względu na przewidywaną czasochłonność operacji z użyciem LLM, system powinien umożliwiać asynchroniczne uruchamianie procesów generowania. Użytkownik powinien mieć możliwość zlecenia zadania, które trafi do kolejki przetwarzania, a system musi zapewniać mechanizm podglądu postępu prac w czasie rzeczywistym.

Bezpośrednia integracja z bazami danych

Poza standardowym eksportem do plików płaskich (JSON, CSV) czy eksportu do poleceń INSERT, system powinien oferować funkcję bezpośredniego przesyłania do zewnętrznych baz danych. Aplikacja powinna obsługiwać połączenia z najpopularniejszymi silnikami SQL (PostgreSQL, MySQL, MSSQL, Oracle) i wykonywać operacje typu *batch insert*. Pozwoli to na natychmiastowe wykorzystanie wygenerowanych danych w środowiskach testowych lub deweloperskich bez konieczności ręcznego importu plików.

Zarządzanie projektami i szablonami

Zalogowani użytkownicy powinni mieć możliwość zapisywania zdefiniowanych schematów w chmurze, co pozwoli na ich późniejszą edycję i ponowne wykorzystanie. System powinien również wspierać import i eksport konfiguracji, ułatwiając współdzielenie schematów w zespołach deweloperskich.

4.2 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają atrybuty jakościowe systemu, które są niezbędne do jego wdrożenia w środowisku produkcyjnym, ze szczególnym naciskiem na wydajność, bezpieczeństwo i skalowalność.

Architektura asynchroniczna i wydajność

System musi być zdolny do obsługi długotrwałych procesów generowania danych bez blokowania interfejsu użytkownika. W tym celu należy zastosować architekturę opartą na kolejce komunikatów, która odseparuje warstwę API od warstwy wykonawczej. Rozwiązanie to powinno zapewniać stabilność działania nawet przy generowaniu zbiorów liczących dziesiątki tysięcy rekordów, a także odporność na chwilowe błędy dostępności API zewnętrznych.

Prywatność danych i obsługa modeli lokalnych

W odpowiedzi na wymogi bezpieczeństwa, system powinien zapewniać możliwość działania w trybie całkowicie odciętym od sieci zewnętrznej [**air_gap_networking**]. Wymagane jest umożliwienie integracji z lokalnymi modelami LLM, co zagwarantuje, że żadne dane ani prompty nie opuszczą infrastruktury wewnętrznej użytkownika.

Skalowalność horyzontalna

System powinien być zaprojektowany z myślą o skalowaniu horyzontalnym, wykorzystując konteneryzację. Pozwoli to na łatwe dostosowanie zasobów obliczeniowych do rosnącego zapotrzebowania, poprzez uruchamianie dodatkowych instancji mikroserwisów w środowiskach chmurowych lub klastrach kontenerowych.

Bezpieczeństwo dostępu

Dostęp do zasobów systemu musi być chroniony mechanizmem uwierzytelniania. Hasła użytkowników powinny być przechowywane w bazie danych wyłącznie w formie zaszyfrowanej, a cała komunikacja między komponentami systemu oraz z zewnętrznymi bazami danych powinna odbywać się przy użyciu bezpiecznych protokołów.

Rozszerzalność i modularność

Architektura systemu powinna zostać zaprojektowana w sposób modułowy. Ma to na celu umożliwienie łatwego dodawania nowych konektorów baz danych oraz integrację z nowymi dostawcami modeli AI w przyszłości, bez konieczności gruntownej przebudowy rdzenia aplikacji.

Rozdział 5

Metodyka pracy

Rozdział ten poświęcony jest metodyce pracy przyjętej podczas realizacji niniejszej pracy dyplomowej. Znajdują się w nim informacje o narzędziach wspomagających proces twórczy, ze szczególnym uwzględnieniem wykorzystania sztucznej inteligencji, a także wyszczególnienie samodzielnego wkładu autora. Ma to na celu transparentne przedstawienie sposobu, w jaki powstał opisany system oraz rzetelne oddzielenie zautomatyzowanej pomocy od pracy własnej dyplomanta.

5.1 Wykorzystanie wsparcia sztucznej inteligencji

Złożoność oraz rozległy zakres technologiczny projektu - obejmujący zarówno tworzenie interfejsu użytkownika, jak i logiki po stronie serwera - skłoniły dyplomanta do zastosowania nowoczesnych narzędzi opartych na sztucznej inteligencji w procesie programistycznym. Wsparcie to obejmowało korzystanie z tradycyjnych asystentów AI, jak również autonomicznych agentów AI wykonujących powierzone sekwencje zadań technicznych (takich jak analiza kodu w wielu plikach nawzajem). Podczas tworzenia wykorzystywane były w szczególności modele z rodziny Gemini (wersje 3 oraz 3.1 Pro).

Zastosowanie tych technologii pozwoliło na przyspieszenie powtarzalnych operacji i szybsze odnajdywanie specyficznych błędów w kodzie, w związku z czym modele te pełniły funkcję wirtualnego asystenta. Głównymi obszarami, w których posłużono się generatywną sztuczną inteligencją, były:

- **Generowanie kodu pomocniczego [boilerplate_code]:** Zautomatyzowanie tworzenia powtarzalnych i nużących struktur, deklaracji podstawowych widoków frontendowych czy bazowych szablonów dla plików konfiguracyjnych.
- **Debugowanie:** Skonsultowanie trudnych do zinterpretowania logów, ostrzeżeń z bibliotek czy błędów pojawiających się w konsoli, co znacząco zmniejszyło czas potrzebny na lokalizację usterek.
- **Konsultacje technologiczne i dokumentacyjne:** Szybkie odszukiwanie właściwych metod w nowych bibliotekach czy przypominanie rzadko używanych fragmentów składni.

5.2 Wkład autorski i decyzyjność

Mimo wsparcia ze strony narzędzi AI, należy podkreślić, że stanowiły one wyłącznie rozwiązanie wspomagające. Cały system w swoim ostatecznym kształcie jest wynikiem samodzielnej pracy koncepcyjnej, badawczej i programistycznej dyplomanta.

Do kluczowych zadań inżynierskich, zrealizowanych całkowicie samodzielnie przez autora, należą:

- **Projektowanie architektury:** Samodzielne zaprojektowanie podziału na logiczne warstwy usług, wybór mechanizmów komunikacji między mikroserwisami (Representational State Transfer (REST) API, WebSocket) oraz decyzyjność co do stosu technologicznego (FastAPI, Redis, Celery, React, baza PostgreSQL).
- **Implementacja silnika generowania danych:** Zaprojektowanie rdzenia logiki backendowej odpowiadającej za walidację oraz generację syntetycznych danych. Wyróżnia się tu m.in. implementację algorytmu sortowania topologicznego rozwiązującego zależności pomiędzy rozbudowanymi relacyjnie tabelami baz danych.
- **Projekt interfejsu i interakcji użytkownika (User Interface (UI)/User Experience (UX)):** Opracowanie unikalnej koncepcji użytej aplikacji webowej i wizualizacji układu paneli roboczych, która musiała zapewniać ergonomię obsługi.
- **Integracja systemu i ocena kodu:** Każdy fragment kodu zaproponowany przez narzędzia AI weryfikowany był przez autora pod kątem poprawności i zgodności z ogólną koncepcją systemu.

Wykorzystanie nowoczesnych asystentów programistycznych znacząco odciążało dyplomanta z najbardziej mechanicznych oraz trywialnych aspektów pisania kodu oprogramowania. Zaoszczędzony w ten sposób czas pozwolił na znacznie głębsze skupienie się na meritum pracy: rozwiązywaniu problemów architektonicznych, a także dopracowywaniu detali implementacyjnych.

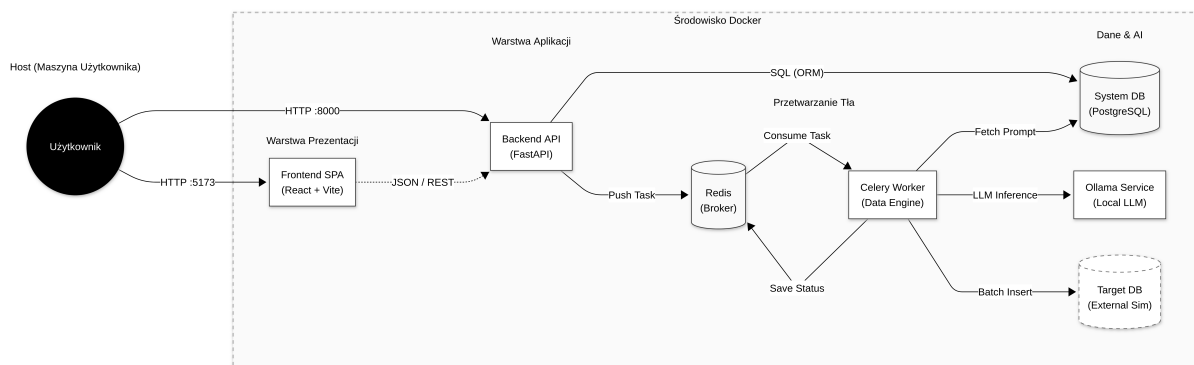
Rozdział 6

Implementacja

W niniejszym rozdziale przedstawiono szczegóły implementacyjne zrealizowanego systemu generatora danych syntetycznych. Opisano architekturę rozwiązania, zastosowane technologie oraz kluczowe komponenty zarówno po stronie serwera (backend), jak i klienta (frontend). Przedstawiono również sposób konfiguracji środowiska uruchomieniowego oraz proces wdrażania aplikacji przy użyciu konteneryzacji.

6.1 Architektura systemu

System został zaprojektowany w architekturze klient-serwer, z wyraźnym podziałem na warstwę prezentacji (frontend) oraz warstwę logiki biznesowej i dostępu do danych (backend). Komunikacja między tymi warstwami odbywa się za pośrednictwem interfejsu REST API [rest_fielding]. Całość systemu została skonteneryzowana przy użyciu platformy Docker, co zapewnia łatwość wdrożenia i spójność środowiska uruchomieniowego. Poniżej przedstawiono schemat architektury systemu (Rysunek 1).



Rysunek 1. Schemat architektury systemu

Główne elementy systemu to:

- **Backend API:** Serwer aplikacji odpowiedzialny za logikę biznesową, autentykację użytkowników oraz zarządzanie procesem generowania danych.
- **Frontend:** Aplikacja internetowa umożliwiająca użytkownikom interakcję z systemem, definiowanie schematów danych oraz podgląd wyników.
- **Baza Danych Aplikacji (PostgreSQL):** Przechowuje informacje o użytkownikach, projektach oraz historię zadań.
- **Kolejka Zadań (Redis):** Pośredniczy w przekazywaniu zadań generowania danych do procesów roboczych (workerów), co pozwala na asynchroniczne przetwarzanie długotrwałych operacji.
- **Worker (Celery):** Proces odpowiedzialny za właściwe generowanie danych na podstawie zdefiniowanych reguł.
- **Ollama (LLM):** Lokalny serwer modeli językowych, wykorzystywany do generowania danych wymagających kreatywności i kontekstu [ollama].

6.2 Implementacja backendowa

Warstwa backendowa została zaimplementowana w języku Python, z wykorzystaniem frameworka FastAPI. Wybór ten podyktowany był wysoką wydajnością FastAPI, wbudowaną obsługą asynchroniczności oraz automatycznym generowaniem dokumentacji API (Swagger UI).

Struktura projektu

Kod źródłowy backendu został podzielony na moduły zgodnie z zasadą pojedynczej odpowiedzialności:

- `main.py`: Punkt wejścia aplikacji, konfiguracja serwera i definicja tras API.
- `models.py`: Modele danych Pydantic służące do walidacji żądań i odpowiedzi API.
- `database.py`: Konfiguracja połączenia z bazą danych przy użyciu biblioteki SQLAlchemy.
- `engine.py`: Główny silnik generowania danych.
- `tasks.py`: Definicje zadań asynchronicznych dla Celery.
- `auth.py`: Logika uwierzytelniania i autoryzacji (OAuth2, haszowanie haseł).

Silnik generowania danych (Data Engine)

Sercem systemu jest klasa `DataEngine` znajdująca się w pliku `engine.py`. Odpowiada ona za interpretację schematu danych zdefiniowanego przez użytkownika i fizyczne wygenerowanie wartości dla poszczególnych pól.

Kluczowym wyzwaniem implementacyjnym było obsłużenie zależności między tabelami (klucze obce). W tym celu zaimplementowano metodę `_resolve_generation_order`, która analizuje graf zależności między tabelami i ustala poprawną kolejność generowania. Algorytm ten zapewnia, że dane do tabeli nadrzędnej są generowane przed danymi do tabeli podrzędnej.

Generowanie wartości dla poszczególnych pól realizowane jest przez dedykowane metody:

- `_generate_template_value`: Pozwala na tworzenie wartości pochodnych przy użyciu szablonów Jinja2, bazując na innych polach w tej samej tabeli.
- `_generate_regex_value`: Generuje ciągi znaków pasujące do podanego wyrażenia regularnego, wykorzystując bibliotekę `rstr`.
- `_generate_timestamp_value`: Generuje losowe daty i czasy w określonym przedziale.
- `_generate_integer_or_float_value`: Generuje liczby całkowite lub zmiennoprzecinkowe w zadanym zakresie.
- `_generate_boolean_value`: Zwraca wartość logiczną prawda/fałsz z określonym prawdopodobieństwem.
- `_generate_faker_value`: Wykorzystuje bibliotekę `Faker` do tworzenia realistycznych danych (imiona, adresy, email).
- `_generate_distribution_value`: Losuje wartość z podanej listy opcji z uwzględnieniem rozkładu prawdopodobieństwa.
- `_generate_foreign_key_value`: Rozwiązuje relacje między tabelami, pobierając identyfikator z innej, wcześniej wygenerowanej tabeli.
- `_generate_llm_value`: Komunikuje się z modelem językowym (przez OpenAI API lub lokalną instancję Ollama) w celu wygenerowania danych kontekstowych.

Przetwarzanie asynchroniczne

Ze względu na potencjalnie długi czas generowania dużych zbiorów danych, proces ten został odseparowany od głównego wątku obsługi żądań HTTP. Wykorzystano w tym celu bibliotekę Celery oraz bazę Redis jako brokera wiadomości.

Użytkownik, zlecając generowanie danych, wysyła żądanie do endpointu `/generate/async`. System tworzy nowe zadanie, zwraca identyfikator `job_id` i kończy obsługę żądania HTTP. Właściwe generowanie odbywa się w tle przez proces Workera. Status zadania jest monitorowany przez klienta w czasie rzeczywistym za pomocą protokołu WebSocket (endpoint `/ws/jobs/{job_id}`), co pozwala na wyświetlanie paska postępu.

Bezpieczeństwo

System implementuje mechanizm uwierzytelniania oparty na tokenach JSON Web Token (JWT) [[jwt_rf7519](#)]. Hasła użytkowników są bezpiecznie przechowywane w bazie danych w formie zahaszowanej (przy użyciu algorytmu `bcrypt`). Każdy chroniony endpoint API weryfikuje ważność tokenu przed przetworzeniem żądania, co realizowane jest za pomocą mechanizmu *Dependency Injection* w FastAPI.

6.3 Implementacja frontendowa

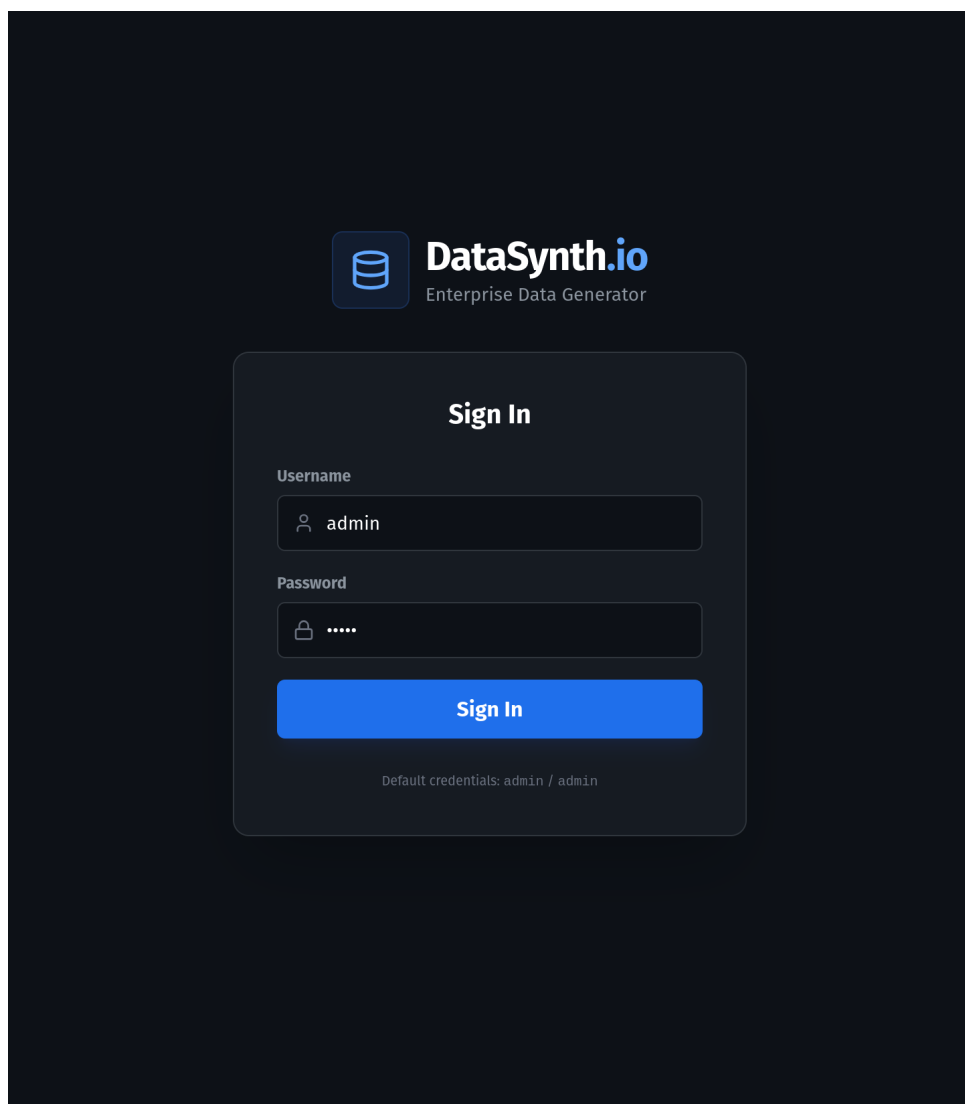
Warstwa prezentacji została zrealizowana jako aplikacja typu Single Page Application (SPA) przy użyciu biblioteki React. Do budowania projektu wykorzystano narzędzie Vite, a za stylowanie odpowiada framework Tailwind Cascading Style Sheets (CSS).

Szczegółowy opis interfejsu

W dalszej części tego rozdziału przedstawiono kluczowe elementy interfejsu graficznego aplikacji. Interfejs został zaprojektowany w sposób minimalistyczny, z wykorzystaniem ciemnego motywu, aby zapewnić komfort pracy przy długotrwałym użytkowaniu. Aplikacja jest w języku angielskim, co jest standardem w narzędziach deweloperskich.

Ekran logowania

Dostęp do systemu jest chroniony i wymaga uwierzytelnienia.

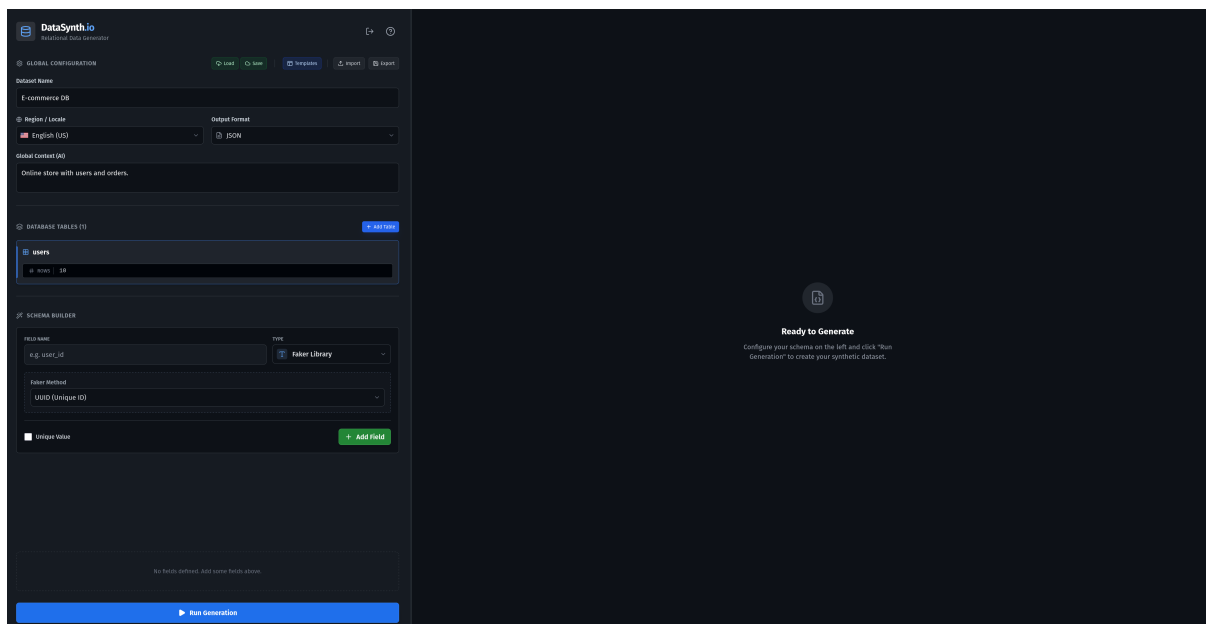


Rysunek 2. Ekran logowania do systemu

Na rysunku 2 przedstawiono widok logowania. Użytkownik musi podać nazwę użytkownika oraz hasło. System komunikuje się z API w celu weryfikacji poświadczeń i w przypadku sukcesu otrzymuje token JWT, który jest zapisywany w pamięci przeglądarki. Interfejs obsługuje również wyświetlanie komunikatów o błędach w przypadku niepoprawnych danych.

Główny widok aplikacji

Po zalogowaniu użytkownik dostaje dostęp do głównego widoku aplikacji, który stanowi centrum tworzenia schematu bazy danych. Interfejs został zaprojektowany z myślą o ergonomii, dzieląc ekran na dwie główne sekcje: lewy panel konfiguracyjny (budowanie schematu) oraz prawy panel wyników (podgląd i eksport).



Rysunek 3. Główny widok aplikacji - edytor schematu i panel wyników

Lewy panel (widoczny na Rysunku 3) zawiera zestaw narzędzi niezbędnych do zdefiniowania struktury zbioru danych. Składa się on z następujących kluczowych modułów:

1. Pasek narzędzi

W górnej części panelu znajduje się pasek narzędzi służący do zarządzania stanem całego projektu. Użytkownik ma tutaj dostęp do następujących funkcjonalności:

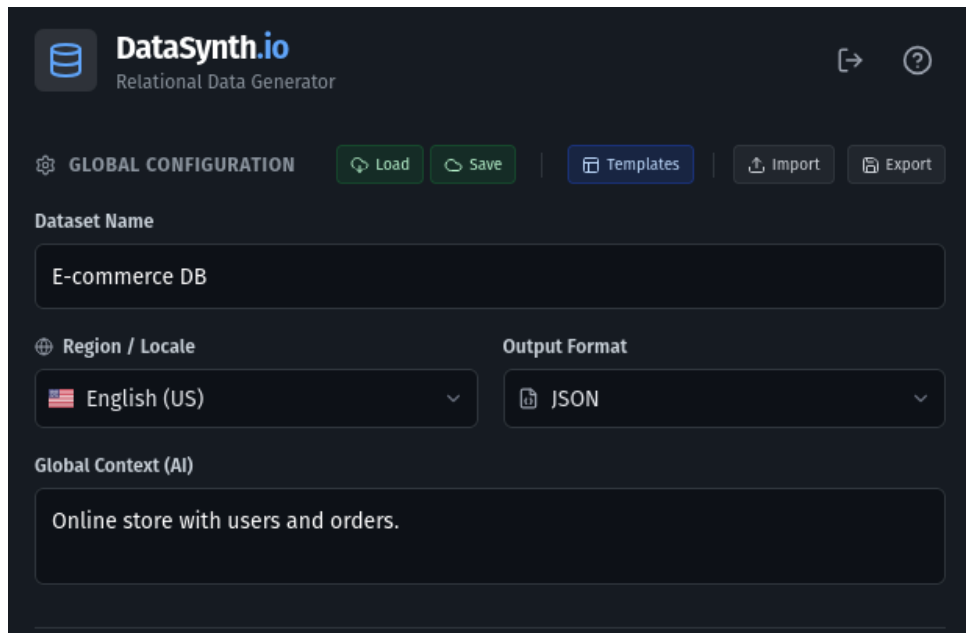
- **Zarządzanie projektami:** Przyciski umożliwiające zapisywanie i wczytywanie projektów z bazy danych.
- **Szablony (Templates):** Dostęp do gotowych, predefiniowanych schematów danych.
- **Import/Eksport:** Możliwość zapisania schematu do lokalnego pliku JSON oraz jego ponownego wczytania.
- **Wczytywanie przykładowych danych:** Opcja pozwalająca na szybkie załadowanie przykładowego schematu (np. „E-commerce”), co jest szczególnie przydatne dla nowych użytkowników.

2. Konfiguracja globalna (Global Configuration)

Poniżej paska narzędzi znajdują się parametry definiujące ogólne właściwości generowanego zbioru danych:

- **Nazwa zbioru:** Pole tekstowe definiujące nazwę zadania, która później służy jako nazwa pliku wynikowego.
- **Lokalizacja (Locale):** Rozwijana lista pozwalająca wybrać krąg kulturowy generowanych danych (np. en_US, pl_PL, de_DE). Wybór ten wpływa na formatowanie dat, walut, numerów telefonów oraz język generowanych tekstów przez bibliotekę Faker.

- **Format wyjściowy:** Pozwala zdefiniować, w jakiej postaci mają zostać zwrócone dane (JSON, CSV spakowany w ZIP, plik SQL).
- **Globalny kontekst AI:** Pole tekstowe, w którym użytkownik może opisać ogólny charakter generowanych danych (np. „System medyczny dla szpitala wojewódzkiego”). Kontekst ten jest „doklejany” do promptów wysyłanych do modeli LLM, co pozwala na zachowanie spójności tematycznej wszystkich generowanych pól.

The image shows the 'GLOBAL CONFIGURATION' section of the DataSynth.io web application. At the top, there's a header with the logo 'DataSynth.io' and the subtitle 'Relational Data Generator'. Below the header, there are several buttons: 'Load', 'Save', 'Templates', 'Import', and 'Export'. The main configuration area includes a 'Dataset Name' field with the value 'E-commerce DB'. Below that, there are two dropdown menus: 'Region / Locale' set to 'English (US)' and 'Output Format' set to 'JSON'. At the bottom, there is a 'Global Context (AI)' text area containing the text 'Online store with users and orders.'.

Rysunek 4. Formularz konfiguracji globalnej

3. Menedżer tabel (Table Manager)

Poniżej konfiguracji znajduje się lista zdefiniowanych tabel. Moduł ten pozwala na:

- **Tworzenie struktur relacyjnych:** Użytkownik może dodawać dowolną liczbę tabel, co pozwala na modelowanie złożonych relacji bazodanowych.
- **Kontrolę wolumenu danych:** Każda tabela posiada niezależny licznik wierszy (Rows), co pozwala np. wygenerować 100 użytkowników i 5000 zamówień.
- **Przeglądanie tabel:** Kliknięcie w nagłówek tabeli aktywuje ją, zmieniając kontekst dla listy pól i edytora.



Rysunek 5. Menedżer tabel

4. Kreator pól (Schema Builder)

Kluczowym elementem interfejsu jest formularz dodawania nowych pól do tabeli.

The image shows a 'SCHEMA BUILDER' interface. It has a main form area and a sidebar. The main form has a 'FIELD NAME' input with the text 'e.g. user_id'. To its right is a 'TYPE' dropdown menu currently showing 'Faker Library'. Below these is a 'Faker Method' dropdown menu showing 'UUID (Unique ID)'. Underneath is a 'Unique Value' checkbox. At the bottom of the main form is a blue button labeled 'Run Generation'. The sidebar on the right lists several categories of data generation: 'AI Generation' (GPT-4o creative content), 'Derived / Logic' (Combine fields (jinja2)), 'Distribution' (Weighted random options), and 'STANDARD DATA'. Under 'STANDARD DATA', there are four options: 'Faker Library' (Real-world data (names, address)), 'Number' (Range of numbers), 'Boolean' (True / False flag), and 'Timestamp' (Date & Time range).

Rysunek 6. Formularz definiowania nowego pola

Moduł ten (Rysunek 6) dynamicznie dostosowuje dostępne parametry w zależności od wybranego typu danych. Użytkownik może:

- Wybrać typ generatora (np. Faker, AI/LLM, Integer, Foreign Key).
- Skonfigurować parametry specyficzne (np. zakres liczb, prompt dla AI, tabelę docelową dla klucza obcego).
- Oznaczyć pole jako unikalne (Unique), co wymusi generowanie niepowtarzalnych wartości.

Na rysunkach 7 oraz 8 przedstawiono przykładowe konfiguracje dla generatora AI oraz generatora z rozkładem prawdopodobieństwa.

The image shows a dark-themed web interface titled "SCHEMA BUILDER". It features a form for defining a new field. At the top, there are two sections: "FIELD NAME" with a text input containing "e.g. user_id", and "TYPE" with a dropdown menu set to "AI Generation". Below these is a dashed box containing configuration options: "Provider" (OpenAI (Cloud)) and "Model" (GPT-4o Mini (Fast)) are both dropdown menus. Underneath is a "Prompt Template" text area with the placeholder "e.g. Generate a name...". Below the prompt template is an "Insert Variable:" section with a button labeled "{id}". Further down is a "Temperature" slider control, currently set to 1.0, with labels "Precise (0.0)", "Creative (1.0)", and "Chaotic (2.0)". At the bottom of the dashed box is a link "Show Advanced Parameters". Below the dashed box, there is a checkbox labeled "Unique Value" and a green button labeled "+ Add Field".

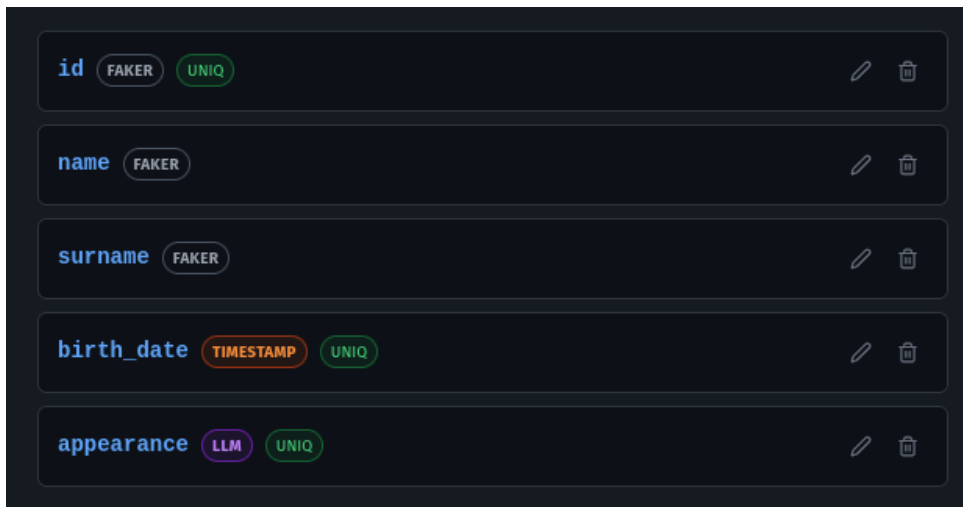
Rysunek 7. Formularz definiowania nowego pola z wykorzystaniem AI

The screenshot shows a 'SCHEMA BUILDER' interface. At the top, there's a 'FIELD NAME' input field containing 'e.g. user_id' and a 'TYPE' dropdown menu set to 'Distribution'. Below this, a dashed box contains a 'Weighted Random' section with a description: 'Define options and their relative probability weights.' Inside this section, there's a table with two columns: 'Option Label' and 'Weight'. It lists 'Option A' and 'Option B', both with a weight of '50 %'. A 'Total: 100.00' indicator is shown at the bottom of the table. To the right of the table is a '+ Add Option' button. Below the dashed box, there's a 'Unique Value' checkbox and a green '+ Add Field' button.

Rysunek 8. Formularz definiowania nowego pola z wykorzystaniem rozkładu prawdopodobieństwa

5. Lista pól (Field List)

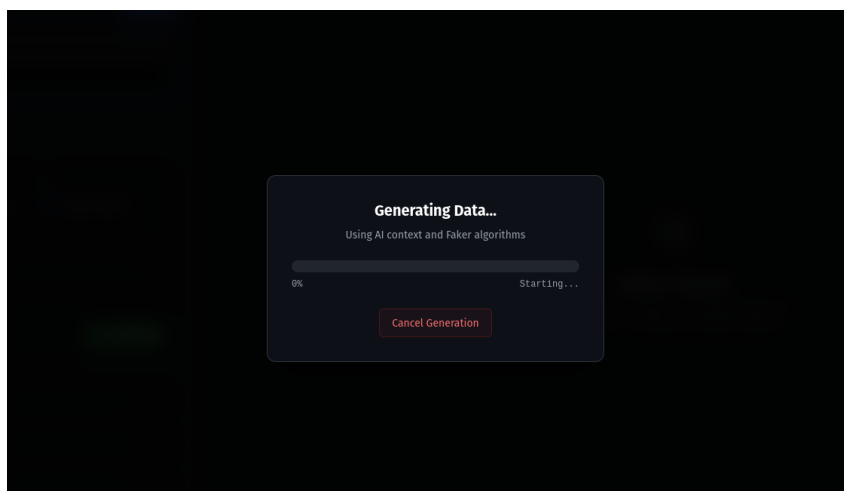
Dolną część panelu zajmuje lista pól dla aktywnej tabeli. Każdy element listy prezentuje nazwę kolumny, typ generatora (oznaczony kolorowym badge'm dla łatwej identyfikacji) oraz ewentualne flagi (np. UNIQ dla wartości unikalnych). W przypadku pól zależnych (np. Template lub Foreign Key), system wizualizuje zależności, wyświetlając informację, do jakich innych atrybutów lub tabel odwołuje się dana kolumna. Moduł umożliwia również szybką edycję parametrów oraz usuwanie poszczególnych pól ze struktury.



Rysunek 9. Lista pól dla aktywnej tabeli

Generowanie i eksport danych

Proces generowania danych odbywa się asynchronicznie, a postęp jest wizualizowany za pomocą paska postępu.

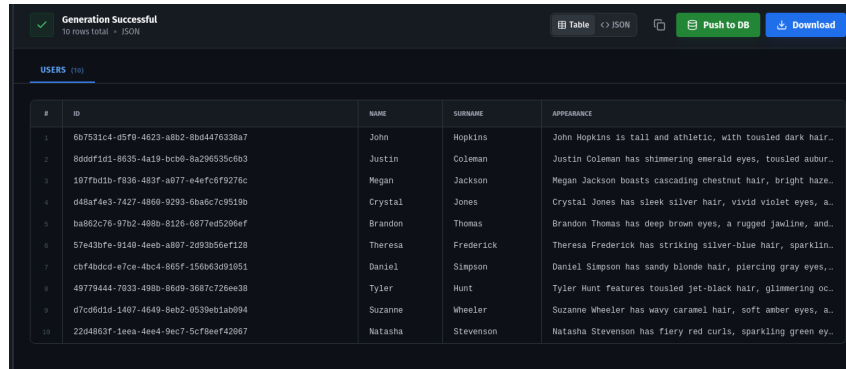


Rysunek 10. Okno postępu generowania

Po zakończeniu procesu (Rysunek 10), wyniki są prezentowane w panelu wyjściowym. Użytkownik ma wtedy możliwość:

- Przeglądania wygenerowanych danych w formacie tabeli, jak i w formacie JSON bezpośrednio w przeglądarce (wprowadzono ograniczenie do 100 wierszy, aby uniknąć problemów z wydajnością).
- Skopiowania widocznego fragmentu danych do schowka.
- Pobrania danych jako plik JSON, SQL lub pakiet CSV.

- Bezpośredniego eksportu danych do zewnętrznej bazy danych (poprzez podanie parametrów połączenia).



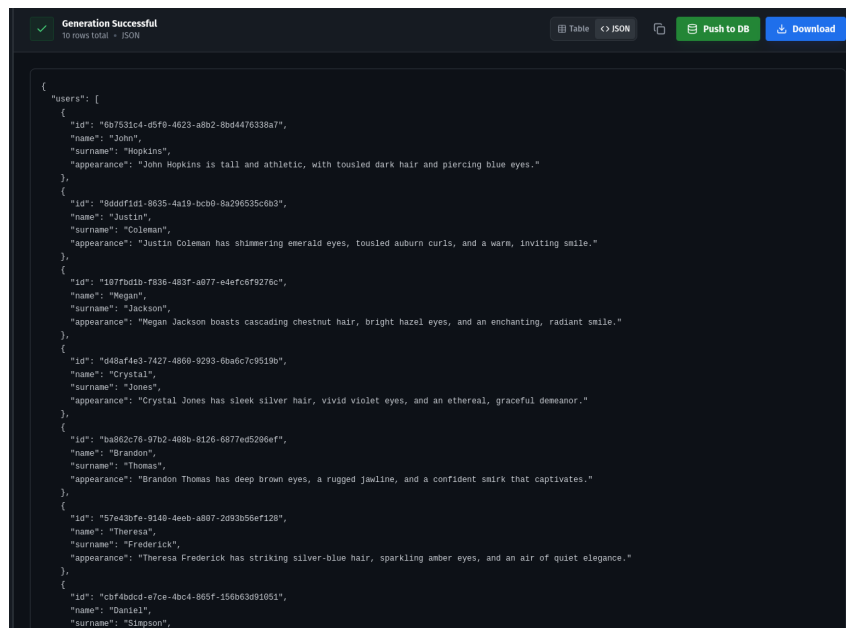
Generation Successful
10 rows total • JSON

Table JSON Push to DB Download

USERS (10)

#	ID	NAME	SURNAME	APPEARANCE
1	6b7531c4-d5f9-4623-abb2-8bd4476338a7	John	Hopkins	John Hopkins is tall and athletic, with tousled dark hair..
2	8dddf1d1-8635-4a19-bcb0-8a296535c6b3	Justin	Coleman	Justin Coleman has shimmering emerald eyes, tousled auburn..
3	187fbd1b-f836-483f-a877-e4efc6f9276c	Megan	Jackson	Megan Jackson boasts cascading chestnut hair, bright hazel..
4	d48af4e3-7427-4860-9293-6ba6c7c9519b	Crystal	Jones	Crystal Jones has sleek silver hair, vivid violet eyes, and..
5	ba862c76-97b2-498b-8126-6877ed5296ef	Brandon	Thomas	Brandon Thomas has deep brown eyes, a rugged jawline, and..
6	57e43bfe-914b-4eeb-ab97-2d93b56ef128	Theresa	Frederick	Theresa Frederick has striking silver-blue hair, sparklin..
7	cbf4bdc4-e7ce-4bc4-865f-156b63d91051	Daniel	Simpson	Daniel Simpson has sandy blonde hair, piercing gray eyes,...
8	49779444-7933-498b-86d9-3687c726ee38	Tyler	Hunt	Tyler Hunt features tousled jet-black hair, glimmering oc..
9	d7c6d1d-1497-4649-8eb2-9539eb1ab094	Suzanne	Wheeler	Suzanne Wheeler has wavy caramel hair, soft amber eyes, a..
10	22d4863f-1eea-4ee4-9ec7-5cf8eef42067	Natasha	Stevenson	Natasha Stevenson has fiery red curls, sparkling green ey..

Rysunek 11. Wygenerowane dane w formacie tabeli

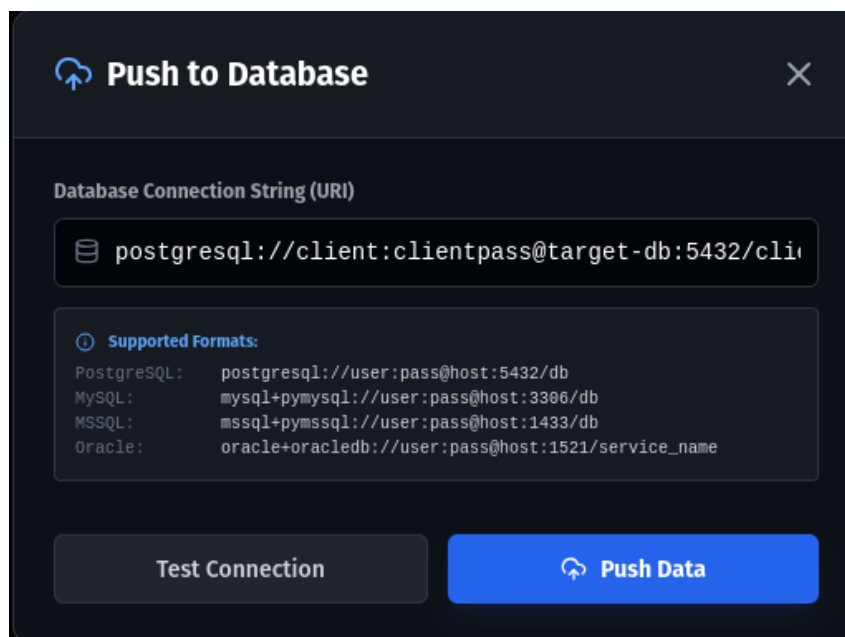


Generation Successful
10 rows total • JSON

Table JSON Push to DB Download

```
{
  "users": [
    {
      "id": "6b7531c4-d5f9-4623-abb2-8bd4476338a7",
      "name": "John",
      "surname": "Hopkins",
      "appearance": "John Hopkins is tall and athletic, with tousled dark hair and piercing blue eyes."
    },
    {
      "id": "8dddf1d1-8635-4a19-bcb0-8a296535c6b3",
      "name": "Justin",
      "surname": "Coleman",
      "appearance": "Justin Coleman has shimmering emerald eyes, tousled auburn curls, and a warm, inviting smile."
    },
    {
      "id": "187fbd1b-f836-483f-a877-e4efc6f9276c",
      "name": "Megan",
      "surname": "Jackson",
      "appearance": "Megan Jackson boasts cascading chestnut hair, bright hazel eyes, and an enchanting, radiant smile."
    },
    {
      "id": "d48af4e3-7427-4860-9293-6ba6c7c9519b",
      "name": "Crystal",
      "surname": "Jones",
      "appearance": "Crystal Jones has sleek silver hair, vivid violet eyes, and an ethereal, graceful demeanor."
    },
    {
      "id": "ba862c76-97b2-498b-8126-6877ed5296ef",
      "name": "Brandon",
      "surname": "Thomas",
      "appearance": "Brandon Thomas has deep brown eyes, a rugged jawline, and a confident smirk that captivates."
    },
    {
      "id": "57e43bfe-914b-4eeb-ab97-2d93b56ef128",
      "name": "Theresa",
      "surname": "Frederick",
      "appearance": "Theresa Frederick has striking silver-blue hair, sparkling amber eyes, and an air of quiet elegance."
    },
    {
      "id": "cbf4bdc4-e7ce-4bc4-865f-156b63d91051",
      "name": "Daniel",
      "surname": "Simpson",
      "appearance": "Daniel Simpson has sandy blonde hair, piercing gray eyes, and a mischievous glint in his blue eyes."
    },
    {
      "id": "49779444-7933-498b-86d9-3687c726ee38",
      "name": "Tyler",
      "surname": "Hunt",
      "appearance": "Tyler Hunt features tousled jet-black hair, glimmering ocean blue eyes, and a rebellious streak."
    },
    {
      "id": "d7c6d1d-1497-4649-8eb2-9539eb1ab094",
      "name": "Suzanne",
      "surname": "Wheeler",
      "appearance": "Suzanne Wheeler has wavy caramel hair, soft amber eyes, and a warm, inviting smile."
    },
    {
      "id": "22d4863f-1eea-4ee4-9ec7-5cf8eef42067",
      "name": "Natasha",
      "surname": "Stevenson",
      "appearance": "Natasha Stevenson has fiery red curls, sparkling green eyes, and a confident, radiant smile."
    }
  ]
}
```

Rysunek 12. Wygenerowane dane w formacie JSON



Rysunek 13. Okno eksportu danych do bazy danych

Na rysunkach 11 oraz 12 przedstawiono przykładowy widok wygenerowanych danych. Użytkownik może łatwo przełączać się między widokiem tabelarycznym a surowym JSON-em. Rysunek 13 prezentuje formularz umożliwiający bezpośredni eksport danych do zewnętrznej bazy danych, co jest szczególnie przydatne dla deweloperów chcących szybko zasilić testową bazę danymi.

Zarządzanie stanem

Stan aplikacji jest zarządzany głównie przy użyciu hooków React (`useState`, `useEffect`). Do obsługi złożonego stanu edytora schematu stworzono dedykowany hook `useSchema`, który centralizuje logikę dodawania, usuwania i edycji tabel oraz pól.

Komunikacja z API

Do komunikacji z backendem wykorzystano bibliotekę `axios`. Zaimplementowano instancję klienta API (w pliku `api.js`), która automatycznie dołącza token autoryzacyjny do każdego żądania oraz obsługuje błędy odpowiedzi serwera (np. automatyczne wylogowanie po wygaśnięciu tokenu).

6.4 Implementacja algorytmów

Algorytm rozwiązywania zależności (sortowanie topologiczne)

Aby zapewnić poprawność relacyjną generowanych danych, niezbędne jest ustalenie odpowiedniej kolejności przetwarzania tabel. Problem ten sprowadza się do posortowania wierzchołków skierowa-

nego grafu zależności w taki sposób, aby dla każdej krawędzi (u, v) wierzchołek u pojawiał się przed wierzchołkiem v .

W systemie zaimplementowano wariant algorytmu Kahna dla sortowania topologicznego [**topological_sort**]. Proces przebiega w następujących krokach:

1. **Budowa grafu zależności:** Dla każdej tabeli analizowanemu są jej pola. Jeśli pole jest typu `foreign_key`, dodawana jest zależność od tabeli docelowej.
2. **Inicjalizacja:** Tworzona jest lista tabel „gotowych” (nieposiadających niewygenerowanych zależności).
3. **Iteracja:** Dopóki lista „gotowych” tabel nie jest pusta:
 - Wybierana jest tabela, dodawana do listy wynikowej.
 - Tabela ta jest usuwana z zależności innych tabel.
 - Jeśli w wyniku usunięcia inna tabela traci wszystkie zależności, trafia na listę „gotowych”.
4. **Weryfikacja:** Jeśli po zakończeniu algorytmu lista wynikowa jest krótsza niż całkowita liczba tabel, oznacza to wykrycie cyklu, co jest zgłaszane jako błąd.

Integracja z LLM

Generowanie danych przy użyciu LLM odbywa się w metodzie `_generate_llm_value`. Proces ten jest znacznie bardziej złożony niż proste wywołanie API.

Konstrukcja promptu

Prompt wysyłany do modelu składa się z trzech części:

- **System Message:** Instruuje model, aby wcielił się w rolę generatora danych fikcyjnych.
- **Szablon użytkownika:** Szablon zdefiniowany przez użytkownika, np. „Opisz osobę {name} {surname}”. Zmienne w nawiasach klamrowych są podmieniane na wartości wygenerowane w bieżącym wierszu przy użyciu silnika Jinja2.
- **Ograniczenia (Constraints):** Dodawane dynamicznie instrukcje, np. wymuszenie unikalności.

Unikalność generowanych danych

Zapewnienie unikalności w modelach probabilistycznych jest wyzwaniem. System stosuje podejście iteracyjne:

```

attempts = 0
while attempts < 10:
    temperature = base_temp + (attempts * 0.1)
    prompt = construct_prompt(template, avoid_values)
    context_injection(prompt, current_row_data)

```

```
value = await call_llm(prompt, temperature)

if is_unique(value):
    return value

attempts += 1
```

Temperatura (*temperature*) jest hiperparametrem kontrolującym losowość generowanych odpowiedzi LLM. Wzrost temperatury w kolejnych próbach spłaszcza rozkład prawdopodobieństwa kolejnych słów, co zwiększa „kreatywność” modelu i znacząco zmniejsza szansę na wygenerowanie duplikatu.

Przetwarzanie wsadowe (Batch Processing)

W celu optymalizacji wydajności generowania danych LLM, zaimplementowano dwufazowy algorytm przetwarzania. W pierwszej fazie silnik generuje wszystkie pola deterministyczne (z parametrem `skip_llm=True`), budując kontekst każdego wiersza. Następnie zebrane konteksty są przekazywane do metody `_generate_llm_batch_value`, która dzieli je na podpartie o stałym rozmiarze (`SUB_BATCH_SIZE`). Dla każdej podpartii formułowany jest zbiorczy prompt, a model jest instruowany, by zwrócił tablicę JSON z odpowiedziami. Podpartie są przetwarzane współbieżnie z ograniczeniem przez `asyncio.Semaphore` (`MAX_CONCURRENT` slotów). Szczegóły procesu optymalizacji i doboru parametrów opisano w Rozdziale 9.4.

6.5 WebSocket

Do wizualizacji postępu w czasie rzeczywistym wykorzystano protokół WebSocket. Po nawiązaniu połączenia pod adresem `ws://server:8001/ws/jobs/{job_id}`, serwer cyklicznie przesyła ramki JSON o strukturze:

```
{
  "job_id": "9fa2204c-72d6-...",
  "status": "processing", // lub "completed", "failed"
  "progress": 45,         // wartość procentowa 0-100
  "total_rows": 1000,
  "error": null           // ewentualny komunikat błędu
}
```

Frontend (komponent `GenerationModal`) subskrybuje te wiadomości i na ich podstawie aktualizuje pasek postępu. Po otrzymaniu statusu `completed`, połączenie jest zamykane, a interfejs przechodzi do widoku wyników.

6.6 Infrastruktura i konteneryzacja

Całe środowisko uruchomieniowe zostało zdefiniowane w pliku `docker-compose.yml`. Definiuje on następujące serwisy:

- `backend`: Kontener z aplikacją FastAPI.
- `frontend`: Kontener serwujący pliki statyczne aplikacji React (lub serwer developerski w trybie dev).
- `worker`: Kontener procesu Celery do zadań w tle.
- `db`: Baza danych PostgreSQL dla aplikacji.
- `redis`: Broker wiadomości dla Celery.
- `target-db`: Dodatkowa instancja PostgreSQL, służąca jako przykładowa docelowa baza danych do bezpośredniego zasilania danymi.
- `ollama`: Usługa serwująca modele LLM.

Dzięki takiemu podejściu, uruchomienie całego systemu sprowadza się do wykonania jednego polecenia `docker compose up`, co znacząco upraszcza proces instalacji i testowania rozwiązania.

Rozdział 7

Podręcznik użytkownika

Niniejszy rozdział stanowi praktyczny przewodnik po systemie DataSynth. Opisano w nim proces instalacji, konfiguracji oraz krok po kroku przedstawiono scenariusz wykorzystania aplikacji do wygenerowania przykładowego zbioru danych.

7.1 Wymagania systemowe i instalacja

Aplikacja została zaprojektowana jako rozwiązanie skonteneryzowane, co minimalizuje wymagania wstępne po stronie systemu operacyjnego hosta.

Wymagania

- System operacyjny: Linux (zalecany Ubuntu 22.04+), Windows 10/11 (z Windows Subsystem for Linux 2 (WSL2)) lub macOS.
- Docker Engine (wersja 24.0+) oraz Docker Compose.
- Min. 4GB pamięci Random Access Memory (RAM) (8GB+ zalecane przy korzystaniu z lokalnych modeli LLM).
- Dostęp do internetu (w celu pobrania obrazów Docker oraz komunikacji z API OpenAI).

Proces instalacji

1. Pobierz kod źródłowy aplikacji z repozytorium.
2. Utwórz w katalogu backend plik `.env` na podstawie dostarczonego szablonu `.env.example`. Uzupełnij klucz API OpenAI (opcjonalnie, jeśli planujesz korzystać z modeli chmurowych).
3. Uruchom terminal w głównym katalogu projektu i wykonaj polecenie:

```
docker compose up -d --build
```

4. Poczekaj, aż wszystkie kontenery (backend, frontend, db, redis, worker) zgłoszą status `healthy`.
5. Otwórz przeglądarkę internetową i przejdź pod adres `http://localhost:5173`.

7.2 Pierwsze kroki: logowanie i interfejs

Przy pierwszym uruchomieniu system automatycznie tworzy konto administratora (login: `admin`, hasło: `admin`). Po zalogowaniu zobaczysz ekran główny.

7.3 Poradnik: generowanie bazy danych „Przychodnia Lekarska”

W tym scenariuszu stworzymy zestaw danych dla systemu obsługi przychodni, składający się z pacjentów, lekarzy i wizyt.

Krok 1: Tabela „Lekarze” (Doctors)

1. Kliknij przycisk „**Add Table**” i nazwij tabelę `doctors`. Ustaw liczbę wierszy na 10.
2. Dodaj pole `id` (typ: `Faker`, metoda: `uuid4`, Unique: `TAK`).
3. Dodaj pole `name` (typ: `Faker`, metoda: `name`).
4. Dodaj pole `specialization` (typ: `Distribution`). W opcjach wpisz: `["Kardiolog", "Pediatria", "Internista"]`, a w wagach: `[20, 30, 50]`.
5. Dodaj pole `bio` (typ: `LLM`). Wpisz prompt: *"Napisz krótką notkę biograficzną (max 2 zdania) dla lekarza o specjalizacji {specialization}."*

Krok 2: Tabela „Pacjenci” (Patients)

1. Dodaj tabelę `patients`. Ustaw liczbę wierszy na 50.
2. Dodaj pola `id`, `first_name`, `last_name`, `email`, `phone` używając odpowiednich metod typu `Faker`.

Krok 3: Tabela „Wizyty” (Appointments)

1. Dodaj tabelę `appointments`. Ustaw liczbę wierszy na 200.
2. Dodaj pole `id`.
3. Dodaj pole `doctor_id` (typ: `Foreign Key`). Wybierz tabelę `doctors` i kolumnę `id`.
4. Dodaj pole `patient_id` (typ: `Foreign Key`). Wybierz tabelę `patients` i kolumnę `id`.

5. Dodaj pole date (typ: Timestamp). Ustaw zakres od -1y do now.
6. Dodaj pole notes (typ: LLM). Prompt: *"Wygeneruj notatkę medyczną z wizyty u lekarza. Pacjent zgłasza typowe objawy."*

Krok 4: Generowanie i eksport

1. Kliknij niebieski przycisk „Run Generation”.
2. Obserwuj pasek postępu. Proces może chwilę potrwać ze względu na generowanie opisów przez LLM.
3. Gdy generowanie się zakończy, sprawdź wyniki w tabeli podglądu. Zwróć uwagę, że doctor_id i patient_id zawsze odpowiadają istniejącym rekordom.
4. Kliknij „Download” aby pobrać gotowy skrypt tworzący i zasilający bazę danych.

7.4 Zaawansowane funkcje

Szablony

Możesz tworzyć pola zależne od innych w tym samym wierszu. Na przykład, aby wygenerować adres email pasujący do imienia i nazwiska, użyj typu Template i wpisz: `{{ first_letter(imie) }}.{{ slugify(nazwisko) }}@firma.com`.

Bezpośredni zrzut do bazy

Jeśli posiadasz uruchomioną bazę danych (np. lokalny PostgreSQL na porcie 5432), możesz użyć przycisku „Push to DB”. Wprowadź adres połączeniowy, np.: `postgresql://user:pass@localhost:5432/mojabaza`. System automatycznie utworzy tabele i wstawi wygenerowane rekordy. Upewnij się, że baza docelowa jest pusta lub nie posiada konfliktów nazw tabel.

Rozdział 8

Testowanie systemu

Proces weryfikacji poprawności działania systemu został podzielony na kilka etapów, obejmujących testy jednostkowe warstwy backendowej, testy integracyjne API oraz manualne testy interfejsu użytkownika. Takie podejście pozwoliło na wykrycie błędów zarówno na poziomie logicznym, jak i funkcjonalnym.

8.1 Testy jednostkowe i integracyjne backendu

Warstwa serwerowa aplikacji, zaimplementowana w języku Python, została pokryta testami automatycznymi przy użyciu frameworka pytest. Głównym celem tych testów była weryfikacja poprawności działania poszczególnych endpointów API oraz logiki biznesowej generatora danych.

Testy, zlokalizowane w katalogu backend/tests, obejmują następujące scenariusze:

- **Weryfikacja autentykacji:** Sprawdzenie poprawności generowania i walidacji tokenów JWT. Testy upewniają się, że dostęp do chronionych zasobów bez ważnego tokenu jest blokowany (kod błędu 401 Unauthorized).
- **Create, Read, Update, Delete (CRUD) projektów:** Testowanie operacji tworzenia, odczytu i usuwania projektów. Weryfikowana jest poprawność zapisu struktury JSON do bazy danych PostgreSQL.
- **Walidacja schematów Pydantic:** Sprawdzenie, czy system poprawnie odrzuca błędne żądania (np. brak wymaganych pól, niepoprawne typy danych).
- **Symulacja generowania danych:** Uruchamianie silnika generującego dla prostych konfiguracji w celu potwierdzenia, że zwracane dane są zgodne z zadeklarowanym typem.

Przykładowy test weryfikujący działanie endpointu /generate wygląda następująco:

```
def test_generate_endpoint(client, auth_headers):
    payload = {
        "config": {"job_name": "Test Job", "rows": 10},
        "tables": [...]
    }
```

```
response = client.post("/generate", json=payload, headers=auth_headers)
assert response.status_code == 200
data = response.json()
assert "status" in data
assert data["status"] == "success"
```

8.2 Testy manualne interfejsu użytkownika

Ze względu na dynamiczny charakter interfejsu, duży nacisk położono na testy manualne. Scenariusze testowe obejmowały pełne User Journeys:

Scenariusz 1: Utworzenie schematu sklepu internetowego

Celem tego testu było sprawdzenie integralności relacyjnej generowanych danych.

1. Utworzenie tabeli users z polami id, name, email.
2. Utworzenie tabeli orders z polem user_id jako klucz obcy do tabeli users.
3. Uruchomienie generowania dla 100 użytkowników i 500 zamówień.
4. **Weryfikacja:** Sprawdzenie, czy każde zamówienie przypisane jest do istniejącego użytkownika. Test zakończył się sukcesem - silnik poprawnie rozwiązał zależności w 100% przypadków.

Scenariusz 2: Integracja z LLM

Testowano zachowanie systemu przy dużym obciążeniu zapytaniami do modelu językowego.

1. Zdefiniowanie pola typu llm z promptem generującym opis produktu.
2. Ustawienie liczby wierszy na 50.
3. **Obserwacja:** Początkowo czas generowania wyniósł średnio 1.66 sekundy na rekord przy użyciu modelu GPT-4o-mini, co przy większych zbiorach danych było wartością niesatysfakcjonującą. W celu optymalizacji wprowadzono mechanizm równoległego przetwarzania zapytań (*batch processing*) z wykorzystaniem `asyncio.gather`. Po tej zmianie czas generowania serii 50 rekordów spadł do 11 sekund (ok. 0.22s na rekord), co stanowi ponad 7-krotne przyspieszenie. System poprawnie obsłużył asynchroniczność, nie blokując interfejsu użytkownika podczas oczekiwania na odpowiedź z API OpenAI.

8.3 Analiza wydajności

Przeprowadzono testy wydajnościowe mające na celu określenie limitów systemu. Testy wykonano na laptopie deweloperskim z kartą graficzną NVIDIA RTX 5060, przy użyciu lokalnego modelu Ollama

(Llama 3). Jako schemat testowy wykorzystano `ecommerce_llm_schema` zawierający 4 tabele, w tym tabelę recenzji z polem `review_text` generowanym przez LLM.

Typ danych	Liczba wierszy	Czas (Faker)	Czas (LLM)
Proste (Imię, Adres)	1 000	0,8s	–
Proste	10 000	5,2s	–
Złożone (z kontekstem AI)	100	–	52,24s (po optymalizacji)
Złożone (z kontekstem AI)	10 000	–	8 038s ($\approx$ 2h 14min)

Tabela 4. Zestawienie czasów generowania danych

Jak widać w Tabeli 4, generowanie danych przy użyciu algorytmów pseudolosowych (Faker) jest niezwykle szybkie i skalowalne liniowo. Wąskim gardłem systemu jest generowanie danych przy użyciu LLM, co potwierdza słuszność decyzji o zastosowaniu przetwarzania wsadowego z kontrolą współbieżności. Szczegółowy opis procesu optymalizacji algorytmu generowania opisano w Rozdziale 9.4.

8.4 Testy End-to-End (E2E)

W celu zapewnienia najwyższej jakości oprogramowania oraz weryfikacji integralności całego systemu, wprowadzono kompleksowe testy typu E2E. Testy te mają na celu symulację rzeczywistych scenariuszy użytkownika aplikacji, sprawdzając współpracę wszystkich komponentów systemu - od interfejsu użytkownika w przeglądarce, przez warstwę API, aż po logikę biznesową i bazy danych.

Do realizacji testów E2E wybrano framework Playwright. Jest on nowoczesnym narzędziem umożliwiającym automatyzację testów w wielu przeglądarkach (Chromium, Firefox, WebKit) oraz zapewniającym wysoką niezawodność dzięki mechanizmom automatycznego oczekiwania na elementy interfejsu.

Zaimplementowany scenariusz testowy pokrywa kluczową ścieżkę krytyczną[**happy_path**] użytkownika:

1. **Autoryzacja:** Scenariusz rozpoczyna się od wejścia na stronę logowania. Automat testowy wprowadza dane uwierzytelniające administratora i weryfikuje poprawne przekierowanie do głównego widoku aplikacji.
2. **Definicja schematu:** Wirtualny użytkownik dodaje nową tabelę o nazwie „users” oraz konfiguruje jej strukturę, dodając kolumnę „name”. Test weryfikuje poprawność interakcji z formularzami oraz dynamiczne aktualizacje interfejsu.
3. **Generowanie danych:** Następuje uruchomienie procesu generowania danych poprzez kliknięcie przycisku „Run Generation”. Test monitoruje asynchroniczny proces, oczekując na komunikaty o postępie przekazywane przez WebSocket.

4. **Weryfikacja wyników:** Ostatnim krokiem jest potwierdzenie zakończenia zadania sukcesem (pojawienie się komunikatu „Data generated!”) oraz sprawdzenie dostępności przycisku pobierania wyników.

Dzięki zastosowaniu symulowania odpowiedzi backendu w niektórych testach, możliwe jest izolowane testowanie warstwy frontendowej, co przyspiesza proces deweloperski i uniezależnia testy interfejsu od dostępności zewnętrznych usług czy modeli LLM. Takie podejście gwarantuje, że błędy w warstwie prezentacji są wykrywane natychmiastowo, zanim kod trafi na środowisko produkcyjne.

8.5 Testy bezpieczeństwa

W celu zapewnienia wysokiego poziomu bezpieczeństwa, przeprowadzono serię testów z wykorzystaniem narzędzi klasy SCA oraz Static Application Security Testing (SAST).

8.5.1 Analiza podatności zależności (SCA)

Do identyfikacji znanych podatności w wykorzystywanych bibliotekach użyto narzędzi `npm audit` (dla warstwy frontendowej) oraz `pip-audit` (dla warstwy backendowej).

- **Frontend:** Analiza początkowo wykazała obecność podatności w bibliotece `axios` (wersja $\leq 1.3.4$). W ramach działań naprawczych, zaktualizowano bibliotekę do najnowszej stabilnej wersji, eliminując zidentyfikowane zagrożenia.
- **Backend:** Audyt zależności Pythonowych nie wykazał krytycznych podatności w głównych bibliotekach produkcyjnych (`FastAPI`, `Uvicorn`, `SQLAlchemy`).

8.5.2 Statyczna analiza kodu (SAST)

Kod źródłowy części serwerowej został poddany analizie przy użyciu narzędzia **Semgrep** [`semgrep`]. Wykryto potencjalne zagrożenie w konfiguracji mechanizmu Cross-Origin Resource Sharing (CORS), polegające na zbyt szerokim uprawnieniu dostępu (`allow_origins=["*"]`). Problem ten został rozwiązany poprzez wprowadzenie konfiguracji opartej na zmiennych środowiskowych, co pozwala na ściśle zdefiniowanie zaufanych domen w środowisku produkcyjnym:

```
allow_origins=os.getenv("ALLOWED_ORIGINS").split(",")
```

Rozdział 9

Wyzwania i problemy implementacyjne

Realizacja systemu łączącego deterministyczne generowanie danych z probabilistyczną naturą modeli językowych wiązała się z szeregiem wyzwań technicznych. W tym rozdziale omówiono napotkane problemy oraz zastosowane rozwiązania.

9.1 Zarządzanie stanem w środowisku asynchronicznym

Jednym z głównych wyzwań było zapewnienie spójności informacji o postępie generowania danych między backendem a frontendem.

Problem

Protokół HTTP jest bezstanowy, co utrudnia przesyłanie informacji o postępie w czasie rzeczywistym. Początkowo rozważano technikę *Long Polling*, jednak przy dużej liczbie jednoczesnych użytkowników generowałoby to nadmiarowy ruch sieciowy.

Rozwiązanie

Zdecydowano się na implementację protokołu WebSocket. Backend uruchamia dedykowany endpoint `/ws/jobs/{job_id}`, który nasłuchuje zdarzeń od menedżera zadań (`job_manager.py`). Worker zapisuje postęp w pamięci współdzielonej (zmienna globalna w instancji serwera lub Redis), a wątek WebSocket cyklicznie (co 0.5s) odczytuje ten stan i wysyła go do klienta. Dzięki temu interfejs użytkownika jest płynny i responsywny.

9.2 Determinizm i unikalność przy zastosowaniu LLM

Modele językowe z natury są niedeterministyczne, co stanowi problem przy generowaniu pól, które muszą być unikalne (np. unikalne opisy produktów, numery seryjne generowane przez AI).

Duplikaty

Podczas testów zauważono, że przy wysokiej temperaturze (parametr `temperature > 1.0`) modele mają tendencję do halucynacji formatu, a przy niskiej (bliskiej 0) często powtarzają te same frazy dla podobnych promptów wejściowych.

Implementacja mechanizmu powień

W klasie `DataEngine` zaimplementowano hybrydowy mechanizm kontroli unikalności:

1. System utrzymuje zbiór wygenerowanych wartości dla każdego pola oznaczonego jako `UNIQUE`.
2. Przy każdej próbie wygenerowania wartości przez LLM, sprawdzane jest, czy wartość już istnieje.
3. W przypadku kolizji, system ponawia próbę (do 10 razy), dynamicznie modyfikując prompt. Do promptu doklejana jest instrukcja: `CONSTRAINT: Value MUST be unique. DO NOT use: [lista ostatnich wartości]`.
4. Dodatkowo, przy każdym ponowieniu delikatnie zwiększana jest temperatura modelu (+0.1), aby wymusić większą różnorodność („kreatywność”) odpowiedzi.

9.3 Rozwiązywanie zależności cyklicznych

Obecna implementacja algorytmu sortowania topologicznego (`_resolve_generation_order`) zakłada, że graf zależności między tabelami jest skierowanym grafem acyklicznym.

W przypadku wystąpienia cyklu (np. Tabela A wymaga Tabeli B, a Tabela B wymaga Tabeli A), system nie jest w stanie ustalić kolejności generowania. W ramach obecnej pracy przyjęto założenie, że użytkownik nie będzie tworzył cyklicznych zależności. W przyszłości planowane jest dodanie walidacji na etapie projektowania schematu, która wykrywałaby cykle i uniemożliwiała zapisanie takiej konfiguracji.

9.4 Optymalizacja wydajności generowania danych z użyciem LLM

Największym wyzwaniem wydajnościowym okazało się generowanie danych z wykorzystaniem lokalnego modelu językowego. W początkowej wersji systemu każde pole typu `llm` było obsługiwane sekwencyjnie – silnik generował jeden wiersz, wysyłał zapytanie do Ollama, oczekiwał na odpowiedź, a dopiero potem przechodził do następnego wiersza. Takie podejście, choć poprawne funkcjonalnie, prowadziło do drastycznie niskiej przepustowości.

Identyfikacja wąskiego gardła

Analiza profilu czasowego wykazała, że generowanie 100 rekordów zawierających pole `llm` (recenzje produktów generowane przez model Llama 3) zajmowało ponad 129 sekund przy podejściu

sekwencyjnym. Czas oczekiwania na odpowiedź modelu stanowił ponad 95% całkowitego czasu generowania, podczas gdy pola deterministyczne (Faker, Regex, Integer) były generowane niemal natychmiast. Każde zapytanie do modelu wymagało ok. 1,3 sekundy, a w trakcie oczekiwania na odpowiedź GPU i CPU pozostawały bezczynne.

Optymalizacja: dwufazowe generowanie z przetwarzaniem wsadowym

W celu maksymalnego wykorzystania zasobów obliczeniowych karty graficznej, zaimplementowano dwufazowy algorytm generowania:

1. **Faza 1 – generowanie pól deterministycznych:** W pierwszej fazie silnik generuje wszystkie wiersze tabeli z pominięciem pól typu `llm` (parametr `skip_llm=True`). Pola deterministyczne (Faker, Regex, Integer, Boolean, Timestamp, Foreign Key, Template, Distribution) są generowane bardzo szybko, tworząc kompletny kontekst wiersza potrzebny do sformułowania promptów LLM.
2. **Faza 2 – wsadowe generowanie pól LLM:** Zebrane konteksty wszystkich wierszy w partii są przekazywane do metody `_generate_llm_batch_value`, która realizuje przetwarzanie wsadowe.

Mechanizm sub-batching z kontrolą współbieżności

Kluczowym elementem optymalizacji jest podział dużej partii wierszy na mniejsze podpartie (*sub-batches*). Każda podpartia zawiera `SUB_BATCH_SIZE` wierszy, dla których formułowany jest jeden zbiorczy prompt. Model LLM jest instruowany, aby zwrócić tablicę JSON zawierającą dokładnie tyle odpowiedzi, ile wierszy zawiera podpartia. Takie podejście pozwala amortyzować narzut związany z inicjalizacją kontekstu modelu.

Dodatkowo, wiele podpartii jest przetwarzanych równolegle z wykorzystaniem mechanizmu `asyncio.Semaphore`, który ogranicza liczbę jednoczesnych zapytań do wartości `MAX_CONCURRENT`. Zapobiega to przeciążeniu serwera Ollama, jednocześnie umożliwiając przetwarzanie potokowe. W przypadku błędu podpartii (np. niepoprawna odpowiedź JSON, niezgodna liczba elementów), system automatycznie stosuje mechanizm awaryjny, przetwarzając wiersze z nieudanej podpartii indywidualnie.

Dostrajanie parametrów

W celu znalezienia optymalnej konfiguracji, przeprowadzono serię testów na schemacie `ecommerce_llm_schema`, zawierającym 4 tabele (łącznie 810 wierszy, w tym 100 recenzji generowanych przez LLM). Testy wykonano na laptopie wyposażonym w kartę graficzną NVIDIA RTX 5060, z użyciem modelu Llama 3 uruchomionego lokalnie w kontenerze Ollama.

Próba	Czas [s]	Czas/wiersz LLM [s]	Opis konfiguracji
1	129,52	1,30	Generowanie sekwencyjne
2	218,88	2,19	Nieoptymalna konfiguracja wsadowa
3	63,43	0,63	Wprowadzenie sub-batching
4	124,41	1,24	Testowanie alternatywnych parametrów
5	52,24	0,52	Optymalna konfiguracja

Tabela 5. Wyniki optymalizacji generowania 100 wierszy LLM (RTX 5060 Laptop)

Jak widać w Tabeli 5, optymalna konfiguracja (próba 5) pozwoliła na osiągnięcie czasu 52,24 s, co stanowi **2,5-krotne przyspieszenie** względem podejścia sekwencyjnego (129,52 s). Warto zauważyć, że próba 2 wykazała regresję wydajności – zbyt agresywna równoległość bez odpowiednich mechanizmów kontroli przepływu prowadziła do przeciążenia serwera Ollama.

Skalowanie do dużych zbiorów danych

Aby zweryfikować skalowalność rozwiązania, przeprowadzono test generowania 10 000 wierszy LLM (łącznie 10 710 wierszy w schemacie) z optymalną konfiguracją parametrów.

Konfiguracja	Wiersze LLM	Czas [s]	Czas/wiersz [s]
Optymalna (RTX 5060 Laptop)	100	52,24	0,52
Optymalna (RTX 5060 Laptop)	10 000	8 038,12	0,80
Optymalna (RTX 4060 Desktop)	100	–	–
Optymalna (RTX 4060 Desktop)	10 000+	–	–

Tabela 6. Porównanie skalowalności generowania LLM na różnych konfiguracjach sprzętowych

Generowanie 10 000 recenzji zajęło 8 038 sekund (ok. 2 godziny 14 minut). Średni czas na wiersz wzrósł z 0,52 s (100 wierszy) do 0,80 s (10 000 wierszy), co sugeruje lekki wzrost narzutu przy dłuższych sesjach inferencji – prawdopodobnie spowodowany rosnącym zużyciem pamięci GPU i termicznym throttlingiem laptopa. Mimo to, system skutecznie przetworzył cały zbiór bez awarii ani błędów krytycznych.

9.5 Obsługa błędów sieciowych

Integracja z API modeli językowych wiąże się z ryzykiem błędów sieciowych (timeouty, błędy 5xx po stronie dostawcy). Aby zapobiec przerwaniu całego procesu generowania przez błąd jednego rekordu, zastosowano bloki try-except wewnątrz pętli generującej. W przypadku błędu API, system przerywa próbę dla tego konkretnego wiersza i wstawia tekst błędu (np. Ollama Error: timeout) jako wartość pola, a następnie kontynuuje pracę nad kolejnymi wierszami. Dzięki temu błąd pojedynczego zapytania nie przerywa wielogodzinnego zadania. Użytkownik widzi te błędy w wynikowym zbiorze

danych i po eksporcie może je odfiltrować z końcowego pliku, bądź – jeżeli skala błędów jest zbyt duża – uruchomić generowanie całego zadania ponownie.

Rozdział 10

Podsumowanie i wnioski

Celem niniejszej pracy magisterskiej było zaprojektowanie i zaimplementowanie nowoczesnego narzędzia do generowania syntetycznych danych relacyjnych, wspieranego przez modele sztucznej inteligencji. Cel ten został osiągnięty poprzez stworzenie kompletnej aplikacji webowej, która integruje szybkość tradycyjnych generatorów z kreatywnością modeli LLM.

10.1 Osiągnięcia

W ramach prac zrealizowano wszystkie założone wymagania funkcjonalne i нефункционалne:

- **Hybrydowy silnik generujący:** Stworzono elastyczny mechanizm łączący bibliotekę Faker, szablony Jinja2 oraz modele LLM (OpenAI/Ollama). Pozwala to na generowanie danych o dowolnym stopniu skomplikowania – od prostych typów liczbowych po złożone opisy tekstowe.
- **Obsługa relacji:** Zaimplementowano algorytm automatycznego rozwiązywania zależności między tabelami, co gwarantuje spójność referencyjną generowanych zbiorów danych.
- **Wydajność:** Dzięki zastosowaniu architektury asynchronicznej (Celery + Redis), system jest w stanie obsłużyć generowanie tysięcy rekordów bez blokowania interfejsu użytkownika.
- **Konteneryzacja:** Całość rozwiązania została zamknięta w kontenerach Docker, co drastycznie upraszcza proces wdrożenia i konfiguracji w dowolnym środowisku.

10.2 Dalszy rozwój aplikacji

Zrealizowany projekt stanowi solidną podstawę do dalszego rozwoju. Zidentyfikowano kilka obszarów, w których system mógłby zostać rozbudowany:

1. **Wsparcie dla kolejnych silników bazodanowych:** Obecnie system wspiera eksport do PostgreSQL. Rozszerzenie o natywne wsparcie dla MySQL, Oracle oraz baz NoSQL (np. MongoDB) zwiększyłoby uniwersalność narzędzia.
2. **Generowanie na podstawie istniejącej bazy:** Ciekawą funkcjonalnością byłaby inżynieria wsteczna – podłączenie się do istniejącej bazy danych, automatyczne odczytanie jej schematu

i wygenerowanie danych testowych „podobnych” do danych produkcyjnych (z zachowaniem anonimizacji).

3. **Inteligentne wykrywanie typów:** Wykorzystanie LLM do automatycznej sugestii typów danych na podstawie samej nazwy kolumny (np. wpisując nazwę "pesel", system sam proponuje odpowiedni generator regex).

Podsumowując, stworzone narzędzie wypełnia lukę na rynku generatorów danych testowych, oferując unikalną możliwość generowania treści semantycznie poprawnych i kontekstowych, co jest kluczowe w dobie nowoczesnych aplikacji opartych na danych.

Wykaz skrótów i symboli

API Application Programming Interface 15, 21, 24, 25, 26, 27, 29, 38, 39, 43, 47, 48, 49, 54

CORS Cross-Origin Resource Sharing 50

CRUD Create, Read, Update, Delete 47

CSS Cascading Style Sheets 28

CSV Comma-Separated Values 11, 20, 31, 36

DOM Document Object Model 17

E2E End-to-End 8, 49

HMR Hot Module Replacement 17

HTTP Hypertext Transfer Protocol 17, 27, 51

JSON JavaScript Object Notation 11, 12, 17, 20, 30, 31, 36, 37, 38, 40, 47, 53, 61

JWT JSON Web Token 27, 29, 47

LLM Large Language Model 9, 12, 13, 15, 16, 17, 18, 19, 20, 21, 26, 31, 33, 39, 40, 41, 43, 44, 45, 49, 50, 52, 53, 54, 57, 58

RAM Random Access Memory 43

REST Representational State Transfer 24, 25

SaaS Software as a Service 11

SAST Static Application Security Testing 50

SCA Software Composition Analysis 8, 50

SPA Single Page Application 16, 28

SQL Structured Query Language 11, 12, 17, 20, 31, 36

UI User Interface 24, 26

UUID Universally Unique Identifier 19

UX User Experience 24

WSL2 Windows Subsystem for Linux 2 43

Spis rysunków

1	Schemat architektury systemu	25
2	Ekran logowania do systemu	29
3	Główny widok aplikacji - edytor schematu i panel wyników	30
4	Formularz konfiguracji globalnej	31
5	Menedżer tabel	32
6	Formularz definiowania nowego pola	33
7	Formularz definiowania nowego pola z wykorzystaniem AI	34
8	Formularz definiowania nowego pola z wykorzystaniem rozkładu prawdopodobieństwa	35
9	Lista pól dla aktywnej tabeli	36
10	Okno postępu generowania	36
11	Wygenerowane dane w formacie tabeli	37
12	Wygenerowane dane w formacie JSON	37
13	Okno eksportu danych do bazy danych	38

Spis tabel

1	Porównanie narzędzi do generowania danych	13
2	Porównanie frameworków webowych Python	16
3	Porównanie technologii frontendowych	16
4	Zestawienie czasów generowania danych	49
5	Wyniki optymalizacji generowania 100 wierszy LLM (RTX 5060 Laptop)	54
6	Porównanie skalowalności generowania LLM na różnych konfiguracjach sprzętowych .	54

Spis załączników